

MyFlowSim

1.1

Generated by Doxygen 1.17.0

1 MyVensim	1
1.1 Descrição	1
1.2 Estrutura do Projeto	1
1.3 Compilação e Execução	2
1.4 Testes	2
1.4.1 Unitários	2
1.4.2 Funcionais	2
1.5 Licença	2
2 Directory Hierarchy	3
2.1 Directories	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Data Structure Index	7
4.1 Data Structures	7
5 File Index	9
5.1 File List	9
6 Directory Documentation	11
6.1 test/functional Directory Reference	11
6.2 include Directory Reference	11
6.3 src Directory Reference	12
6.4 test Directory Reference	12
6.5 test/unit Directory Reference	13
7 Data Structure Documentation	15
7.1 Flow Class Reference	15
7.1.1 Detailed Description	16
7.1.2 Constructor & Destructor Documentation	16
7.1.2.1 ~Flow()	16
7.1.3 Member Function Documentation	17
7.1.3.1 execute()	17
7.1.3.2 getName()	17
7.1.3.3 getSource()	17
7.1.3.4 getTarget()	17
7.1.3.5 setName()	17
7.1.3.6 setSource()	17
7.1.3.7 setTarget()	18
7.2 Flow_Impl Class Reference	18
7.2.1 Constructor & Destructor Documentation	21
7.2.1.1 Flow_Impl() [1/3]	21
7.2.1.2 Flow_Impl() [2/3]	22

7.2.1.3 Flow_Impl() [3/3]	22
7.2.1.4 ~Flow_Impl()	23
7.2.2 Member Function Documentation	23
7.2.2.1 execute()	23
7.2.2.2 getName()	23
7.2.2.3 getSource()	24
7.2.2.4 getTarget()	24
7.2.2.5 operator=()	24
7.2.2.6 setName()	25
7.2.2.7 setSource()	25
7.2.2.8 setTarget()	26
7.2.3 Field Documentation	26
7.2.3.1 name	26
7.2.3.2 source	26
7.2.3.3 target	26
7.3 Flow_Test Class Reference	26
7.3.1 Detailed Description	29
7.3.2 Constructor & Destructor Documentation	29
7.3.2.1 Flow_Test() [1/2]	29
7.3.2.2 Flow_Test() [2/2]	30
7.3.3 Member Function Documentation	30
7.3.3.1 execute()	30
7.4 FlowComplexo Class Reference	31
7.4.1 Detailed Description	33
7.4.2 Constructor & Destructor Documentation	34
7.4.2.1 FlowComplexo() [1/3]	34
7.4.2.2 FlowComplexo() [2/3]	34
7.4.2.3 FlowComplexo() [3/3]	34
7.4.2.4 ~FlowComplexo()	35
7.4.3 Member Function Documentation	35
7.4.3.1 execute()	35
7.4.3.2 operator=()	35
7.5 FlowExponencial Class Reference	36
7.5.1 Detailed Description	39
7.5.2 Constructor & Destructor Documentation	40
7.5.2.1 FlowExponencial() [1/3]	40
7.5.2.2 FlowExponencial() [2/3]	40
7.5.2.3 FlowExponencial() [3/3]	40
7.5.2.4 ~FlowExponencial()	41
7.5.3 Member Function Documentation	41
7.5.3.1 execute()	41
7.5.3.2 operator=()	41
7.6 FlowLogistico Class Reference	42

7.6.1 Detailed Description	45
7.6.2 Constructor & Destructor Documentation	46
7.6.2.1 FlowLogistico() [1/3]	46
7.6.2.2 FlowLogistico() [2/3]	46
7.6.2.3 FlowLogistico() [3/3]	46
7.6.2.4 ~FlowLogistico()	47
7.6.3 Member Function Documentation	47
7.6.3.1 execute()	47
7.6.3.2 operator=()	47
7.7 FlowTest Class Reference	48
7.7.1 Detailed Description	51
7.7.2 Constructor & Destructor Documentation	51
7.7.2.1 FlowTest() [1/2]	51
7.7.2.2 FlowTest() [2/2]	52
7.7.3 Member Function Documentation	52
7.7.3.1 execute()	52
7.8 Model Class Reference	52
7.8.1 Detailed Description	54
7.8.2 Constructor & Destructor Documentation	54
7.8.2.1 ~Model()	54
7.8.3 Member Function Documentation	54
7.8.3.1 add() [1/2]	54
7.8.3.2 add() [2/2]	54
7.8.3.3 run()	54
7.8.3.4 showModel()	55
7.9 Model_Impl Class Reference	55
7.9.1 Detailed Description	58
7.9.2 Constructor & Destructor Documentation	58
7.9.2.1 Model_Impl() [1/2]	58
7.9.2.2 ~Model_Impl()	58
7.9.2.3 Model_Impl() [2/2]	58
7.9.3 Member Function Documentation	59
7.9.3.1 add() [1/2]	59
7.9.3.2 add() [2/2]	59
7.9.3.3 operator=()	60
7.9.3.4 run()	61
7.9.3.5 showModel()	62
7.9.4 Field Documentation	62
7.9.4.1 flows	62
7.9.4.2 systems	62
7.10 System Class Reference	63
7.10.1 Detailed Description	64
7.10.2 Constructor & Destructor Documentation	64

7.10.2.1 <code>~System()</code>	64
7.10.3 Member Function Documentation	64
7.10.3.1 <code>getName()</code>	64
7.10.3.2 <code>getValue()</code>	65
7.10.3.3 <code>setName()</code>	65
7.10.3.4 <code>setValue()</code>	65
7.11 <code>System_Impl</code> Class Reference	66
7.11.1 Detailed Description	69
7.11.2 Constructor & Destructor Documentation	69
7.11.2.1 <code>System_Impl()</code> [1/3]	69
7.11.2.2 <code>System_Impl()</code> [2/3]	69
7.11.2.3 <code>System_Impl()</code> [3/3]	70
7.11.2.4 <code>~System_Impl()</code>	70
7.11.3 Member Function Documentation	70
7.11.3.1 <code>getName()</code>	70
7.11.3.2 <code>getValue()</code>	70
7.11.3.3 <code>operator=()</code>	71
7.11.3.4 <code>setName()</code>	72
7.11.3.5 <code>setValue()</code>	72
7.11.4 Field Documentation	72
7.11.4.1 <code>name</code>	72
7.11.4.2 <code>value</code>	72
8 File Documentation	75
8.1 <code>include/flow.h</code> File Reference	75
8.1.1 Detailed Description	75
8.2 <code>flow.h</code>	75
8.3 <code>include/flow_impl.h</code> File Reference	76
8.4 <code>flow_impl.h</code>	76
8.5 <code>include/flows.h</code> File Reference	77
8.5.1 Detailed Description	78
8.6 <code>flows.h</code>	78
8.7 <code>include/functional_test.h</code> File Reference	79
8.7.1 Detailed Description	80
8.7.2 Function Documentation	80
8.7.2.1 <code>complexFunctionalTest()</code>	80
8.7.2.2 <code>exponentialFunctionalTest()</code>	81
8.7.2.3 <code>logisticalFunctionalTest()</code>	81
8.8 <code>functional_test.h</code>	82
8.9 <code>include/model.h</code> File Reference	82
8.9.1 Detailed Description	83
8.10 <code>model.h</code>	83
8.11 <code>include/model_impl.h</code> File Reference	83

8.12	model_impl.h	84
8.13	include/system.h File Reference	85
8.13.1	Detailed Description	85
8.14	system.h	85
8.15	include/system_impl.h File Reference	85
8.16	system_impl.h	86
8.17	README.md File Reference	87
8.18	src/flow_impl.cpp File Reference	87
8.18.1	Detailed Description	87
8.19	src/main.cpp File Reference	87
8.19.1	Detailed Description	88
8.19.2	Macro Definition Documentation	88
8.19.2.1	MAIN	88
8.19.3	Function Documentation	88
8.19.3.1	main()	88
8.20	test/functional/main.cpp File Reference	88
8.20.1	Detailed Description	89
8.20.2	Macro Definition Documentation	89
8.20.2.1	MAIN_FUNCIONAL_TESTS	89
8.20.3	Function Documentation	89
8.20.3.1	main()	89
8.21	test/unit/main.cpp File Reference	90
8.21.1	Detailed Description	91
8.21.2	Function Documentation	91
8.21.2.1	main()	91
8.22	src/model_impl.cpp File Reference	92
8.22.1	Detailed Description	93
8.23	src/system_impl.cpp File Reference	93
8.23.1	Detailed Description	93
8.24	test/functional/flows.cpp File Reference	93
8.24.1	Detailed Description	94
8.25	test/functional/functional_test.cpp File Reference	94
8.25.1	Detailed Description	95
8.25.2	Function Documentation	95
8.25.2.1	complexFuncionalTest()	95
8.25.2.2	exponentialFuncionalTest()	96
8.25.2.3	floatingPointComparison()	97
8.25.2.4	logisticalFuncionalTest()	97
8.26	test/unit/unit_flow.cpp File Reference	98
8.26.1	Detailed Description	100
8.26.2	Function Documentation	100
8.26.2.1	run_unit_tests_Flow()	100
8.26.2.2	unit_Flow_constructor()	100

8.26.2.3	unit_Flow_destructor()	101
8.26.2.4	unit_Flow_execute()	101
8.26.2.5	unit_Flow_getName()	102
8.26.2.6	unit_Flow_getSource()	102
8.26.2.7	unit_Flow_getTarget()	103
8.26.2.8	unit_Flow_setName()	103
8.26.2.9	unit_Flow_setSource()	104
8.26.2.10	unit_Flow_setTarget()	105
8.27	test/unit/unit_flow.h File Reference	105
8.27.1	Detailed Description	106
8.27.2	Function Documentation	106
8.27.2.1	run_unit_tests_Flow()	106
8.27.2.2	unit_Flow_constructor()	106
8.27.2.3	unit_Flow_destructor()	107
8.27.2.4	unit_Flow_execute()	107
8.27.2.5	unit_Flow_getName()	108
8.27.2.6	unit_Flow_getSource()	108
8.27.2.7	unit_Flow_getTarget()	109
8.27.2.8	unit_Flow_setName()	109
8.27.2.9	unit_Flow_setSource()	110
8.27.2.10	unit_Flow_setTarget()	111
8.28	unit_flow.h	111
8.29	test/unit/unit_model.cpp File Reference	111
8.29.1	Detailed Description	112
8.29.2	Function Documentation	113
8.29.2.1	run_unit_tests_Model()	113
8.29.2.2	unit_Model_addFlow()	113
8.29.2.3	unit_Model_addSystem()	114
8.29.2.4	unit_Model_constructor()	114
8.29.2.5	unit_Model_destructor()	115
8.29.2.6	unit_Model_run()	115
8.29.2.7	unit_Model_showModel()	116
8.30	test/unit/unit_model.h File Reference	116
8.30.1	Detailed Description	117
8.30.2	Function Documentation	117
8.30.2.1	run_unit_tests_Model()	117
8.30.2.2	unit_Model_addFlow()	117
8.30.2.3	unit_Model_addSystem()	118
8.30.2.4	unit_Model_constructor()	119
8.30.2.5	unit_Model_destructor()	119
8.30.2.6	unit_Model_run()	119
8.30.2.7	unit_Model_showModel()	120
8.31	unit_model.h	120

8.32 test/unit/unit_system.cpp File Reference	121
8.32.1 Detailed Description	122
8.32.2 Function Documentation	122
8.32.2.1 run_unit_tests_System()	122
8.32.2.2 unit_System_constructor()	122
8.32.2.3 unit_System_destructor()	123
8.32.2.4 unit_System_getValue()	123
8.32.2.5 unit_System_setValue()	124
8.33 test/unit/unit_system.h File Reference	124
8.33.1 Detailed Description	125
8.33.2 Function Documentation	125
8.33.2.1 run_unit_tests_System()	125
8.33.2.2 unit_System_constructor()	126
8.33.2.3 unit_System_destructor()	127
8.33.2.4 unit_System_getValue()	127
8.33.2.5 unit_System_setValue()	127
8.34 unit_system.h	128
8.35 test/unit/unit_tests.cpp File Reference	128
8.35.1 Detailed Description	129
8.35.2 Function Documentation	129
8.35.2.1 run_unit_tests_globals()	129
8.36 test/unit/unit_tests.h File Reference	130
8.36.1 Detailed Description	131
8.36.2 Function Documentation	131
8.36.2.1 run_unit_tests_globals()	131
8.37 unit_tests.h	132
Index	133

Chapter 1

MyVensim

Framework em C++ para construção e execução de modelos de simulação baseados em Dinâmica de Sistemas.

Disciplina: BCC322 — Engenharia de Software I · UFOP

Professor: Tiago Garcia de Senna Carneiro

1.1 Descrição

API inspirada na linguagem DYNAMO e no simulador Vensim. Permite modelar e executar sistemas dinâmicos compostos por sistemas (estoques), fluxos (equações de transferência) e modelos (orquestradores da simulação).

1.2 Estrutura do Projeto

```
MyVensim/  
  include/  
    flow.h  
    flow_impl.h  
    flows.h  
    functional_test.h  
    model.h  
    model_impl.h  
    system.h  
    system_impl.h  
  
  src/  
    main.cpp  
    flow_impl.cpp  
    flows.cpp  
    model_impl.cpp  
    system_impl.cpp  
  
  test/  
    functional/  
      main.cpp  
      functional_test.cpp  
    unit/  
      main.cpp  
      unit_tests.h  
      unit_tests.cpp  
      unit_flow.h  
      unit_flow.cpp  
      unit_model.h  
      unit_model.cpp  
      unit_system.h  
      unit_system.cpp  
  
  bin/  
  Makefile  
  README.md
```

1.3 Compilação e Execução

Requer g++ com suporte a C++17.

Comando	Descrição
make	Compila tudo (simulador + testes)
make unit_tests	Compila e executa os testes unitários
make functional_tests	Compila e executa os testes funcionais
make run	Executa testes unitários e funcionais
make clean	Remove binários e objetos gerados

1.4 Testes

1.4.1 Unitários

Verificam individualmente os métodos de [System](#), [Flow](#) e [Model](#).

Executável: bin/progTestUnit

1.4.2 Funcionais

Verificam a simulação completa contra valores analíticos esperados para três modelos:

Modelo	Descrição
Exponencial	Crescimento proporcional ao estoque de origem (taxa 1%)
Logístico	Crescimento com capacidade limite de 70
Complexo	Rede com 5 sistemas e 6 fluxos interligados

Executável: bin/progTestFuncional

1.5 Licença

Desenvolvido exclusivamente para fins acadêmicos na disciplina BCC322 — UFOP.

Uso comercial não permitido sem autorização prévia do autor.

Chapter 2

Directory Hierarchy

2.1 Directories

functional	11
flows.cpp	93
functional_test.cpp	94
main.cpp	88
include	11
flow.h	75
flow_impl.h	76
flows.h	77
functional_test.h	79
model.h	82
model_impl.h	83
system.h	85
system_impl.h	85
src	12
flow_impl.cpp	87
main.cpp	87
model_impl.cpp	92
system_impl.cpp	93
test	12
functional	11
flows.cpp	93
functional_test.cpp	94
main.cpp	88
unit	13
main.cpp	90
unit_flow.cpp	98
unit_flow.h	105
unit_model.cpp	111
unit_model.h	116
unit_system.cpp	121
unit_system.h	124
unit_tests.cpp	128
unit_tests.h	130
unit	13
main.cpp	90
unit_flow.cpp	98
unit_flow.h	105
unit_model.cpp	111
unit_model.h	116
unit_system.cpp	121

unit_system.h	124
unit_tests.cpp	128
unit_tests.h	130

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Flow	15
Flow_Impl	18
FlowComplexo	31
FlowExponencial	36
FlowLogistico	42
FlowTest	48
Flow_Test	26
Model	52
Model_Impl	55
System	63
System_Impl	66

Chapter 4

Data Structure Index

4.1 Data Structures

Here are the data structures with brief descriptions:

Flow	Interface abstrata que representa um fluxo entre dois sistemas	15
Flow_Impl		18
Flow_Test	Implementação concreta de Flow_Impl utilizada nos testes unitários	26
FlowComplexo	Fluxo com equação de múltiplas interações entre sistemas	31
FlowExponencial	Fluxo baseado em crescimento exponencial	36
FlowLogistico	Fluxo baseado em crescimento logístico	42
FlowTest	Implementação concreta de Flow_Impl utilizada nos testes do Model	48
Model	Interface abstrata que representa um modelo de dinâmica de sistemas	52
Model_Impl	Classe responsável por representar um modelo de simulação	55
System	Interface abstrata que representa um sistema em um modelo de simulação	63
System_Impl	Classe responsável por representar um sistema	66

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

include/ flow.h	Declaração da interface abstrata Flow	75
include/ flow_impl.h		76
include/ flows.h	Declaração das classes de fluxo concretas: exponencial, logístico e complexo	77
include/ functional_test.h	Declaração dos testes funcionais dos modelos de simulação	79
include/ model.h	Declaração da interface abstrata Model	82
include/ model_impl.h		83
include/ system.h	Declaração da interface abstrata System	85
include/ system_impl.h		85
src/ flow_impl.cpp	Implementação da classe Flow_Impl	87
src/ main.cpp	Ponto de entrada principal do simulador	87
src/ model_impl.cpp	Implementação da classe Model_Impl	92
src/ system_impl.cpp	Implementação da classe System_Impl	93
test/functional/ flows.cpp	Implementação dos fluxos do simulador	93
test/functional/ functional_test.cpp	Implementação dos testes funcionais do simulador	94
test/functional/ main.cpp	Ponto de entrada dos testes funcionais do simulador	88
test/unit/ main.cpp	Ponto de entrada da suíte de testes unitários	90
test/unit/ unit_flow.cpp	Implementação dos testes unitários da classe Flow	98
test/unit/ unit_flow.h	Declaração dos testes unitários da classe Flow	105
test/unit/ unit_model.cpp	Implementação dos testes unitários da classe Model	111
test/unit/ unit_model.h	Declaração dos testes unitários da classe Model	116
test/unit/ unit_system.cpp	Implementação dos testes unitários da classe System	121

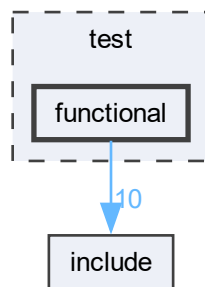
test/unit/ unit_system.h	
Declaração dos testes unitários da classe System	124
test/unit/ unit_tests.cpp	
Implementação da função orquestradora dos testes unitários globais	128
test/unit/ unit_tests.h	
Declaração da função de entrada dos testes unitários globais	130

Chapter 6

Directory Documentation

6.1 test/functional Directory Reference

Directory dependency graph for functional:



Files

- file [flows.cpp](#)
Implementação dos fluxos do simulador.
- file [functional_test.cpp](#)
Implementação dos testes funcionais do simulador.
- file [main.cpp](#)
Ponto de entrada dos testes funcionais do simulador.

6.2 include Directory Reference

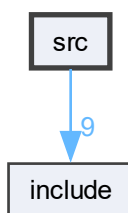
Files

- file [flow.h](#)
Declaração da interface abstrata [Flow](#).
- file [flow_impl.h](#)
- file [flows.h](#)
Declaração das classes de fluxo concretas: exponencial, logístico e complexo.
- file [functional_test.h](#)
Declaração dos testes funcionais dos modelos de simulação.

- file [model.h](#)
Declaração da interface abstrata [Model](#).
- file [model_impl.h](#)
- file [system.h](#)
Declaração da interface abstrata [System](#).
- file [system_impl.h](#)

6.3 src Directory Reference

Directory dependency graph for src:

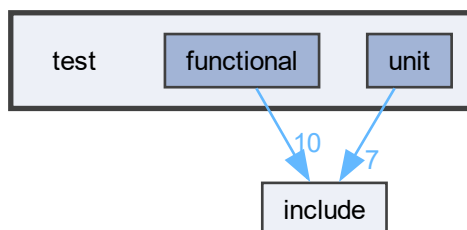


Files

- file [flow_impl.cpp](#)
Implementação da classe [Flow_Impl](#).
- file [main.cpp](#)
Ponto de entrada principal do simulador.
- file [model_impl.cpp](#)
Implementação da classe [Model_Impl](#).
- file [system_impl.cpp](#)
Implementação da classe [System_Impl](#).

6.4 test Directory Reference

Directory dependency graph for test:

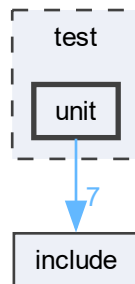


Directories

- directory [functional](#)
- directory [unit](#)

6.5 test/unit Directory Reference

Directory dependency graph for unit:



Files

- file [main.cpp](#)
Ponto de entrada da suíte de testes unitários.
- file [unit_flow.cpp](#)
Implementação dos testes unitários da classe [Flow](#).
- file [unit_flow.h](#)
Declaração dos testes unitários da classe [Flow](#).
- file [unit_model.cpp](#)
Implementação dos testes unitários da classe [Model](#).
- file [unit_model.h](#)
Declaração dos testes unitários da classe [Model](#).
- file [unit_system.cpp](#)
Implementação dos testes unitários da classe [System](#).
- file [unit_system.h](#)
Declaração dos testes unitários da classe [System](#).
- file [unit_tests.cpp](#)
Implementação da função orquestradora dos testes unitários globais.
- file [unit_tests.h](#)
Declaração da função de entrada dos testes unitários globais.

Chapter 7

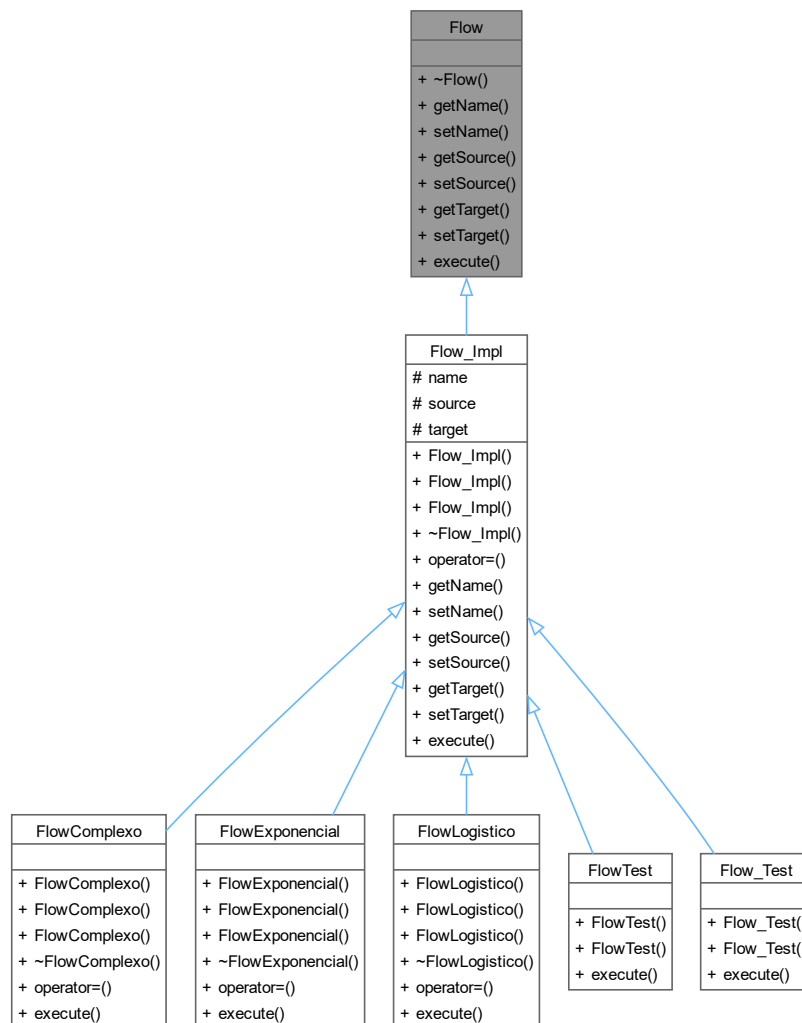
Data Structure Documentation

7.1 Flow Class Reference

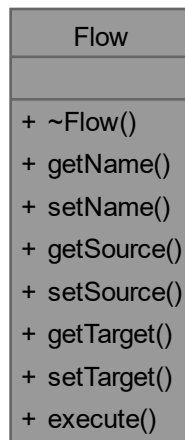
Interface abstrata que representa um fluxo entre dois sistemas.

```
#include <flow.h>
```

Inheritance diagram for Flow:



Collaboration diagram for Flow:



Public Member Functions

- virtual `~Flow()` = default
Destrutor virtual da classe `Flow`.
- virtual `string getName()` const = 0
Retorna o nome do fluxo.
- virtual `void setName(const string &name)` = 0
Define o nome do fluxo.
- virtual `System * getSource()` const = 0
Retorna o sistema de origem do fluxo.
- virtual `void setSource(System *source)` = 0
Define o sistema de origem do fluxo.
- virtual `System * getTarget()` const = 0
Retorna o sistema de destino do fluxo.
- virtual `void setTarget(System *target)` = 0
Define o sistema de destino do fluxo.
- virtual `double execute()` = 0
Executa o cálculo do fluxo para um instante de tempo.

7.1.1 Detailed Description

Interface abstrata que representa um fluxo entre dois sistemas.

A classe `Flow` define a interface para os fluxos de um modelo de dinâmica de sistemas. Um fluxo conecta dois sistemas (origem e destino) e define uma equação que determina a quantidade transferida por unidade de tempo. Subclasses devem implementar o método `execute()`.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 ~Flow()

`virtual Flow::~~Flow()` [virtual], [default]

Destrutor virtual da classe `Flow`.

7.1.3 Member Function Documentation

7.1.3.1 execute()

virtual double Flow::execute () [pure virtual]

Executa o cálculo do fluxo para um instante de tempo.

Método puramente virtual implementado pelas subclasses. Calcula a quantidade a ser transferida do sistema de origem para o sistema de destino com base na equação específica do fluxo.

Returns

Valor calculado pelo fluxo.

Implemented in [Flow_Impl](#), [Flow_Test](#), [FlowComplexo](#), [FlowExponencial](#), [FlowLogistico](#), and [FlowTest](#).

7.1.3.2 getName()

virtual string Flow::getName () const [pure virtual]

Retorna o nome do fluxo.

Returns

Nome do fluxo.

Implemented in [Flow_Impl](#).

7.1.3.3 getSource()

virtual [System](#) * Flow::getSource () const [pure virtual]

Retorna o sistema de origem do fluxo.

Returns

Ponteiro para o sistema de origem.

Implemented in [Flow_Impl](#).

7.1.3.4 getTarget()

virtual [System](#) * Flow::getTarget () const [pure virtual]

Retorna o sistema de destino do fluxo.

Returns

Ponteiro para o sistema de destino.

Implemented in [Flow_Impl](#).

7.1.3.5 setName()

virtual void Flow::setName (
const string & name) [pure virtual]

Define o nome do fluxo.

Parameters

name	Novo nome do fluxo.
------	---------------------

Implemented in [Flow_Impl](#).

7.1.3.6 setSource()

virtual void Flow::setSource (
[System](#) * source) [pure virtual]

Define o sistema de origem do fluxo.

Parameters

source	Ponteiro para o novo sistema de origem.
--------	---

Implemented in [Flow_Impl](#).

7.1.3.7 setTarget()

virtual void Flow::setTarget (
 [System](#) * target) [pure virtual]

Define o sistema de destino do fluxo.

Parameters

target	Ponteiro para o novo sistema de destino.
--------	--

Implemented in [Flow_Impl](#).

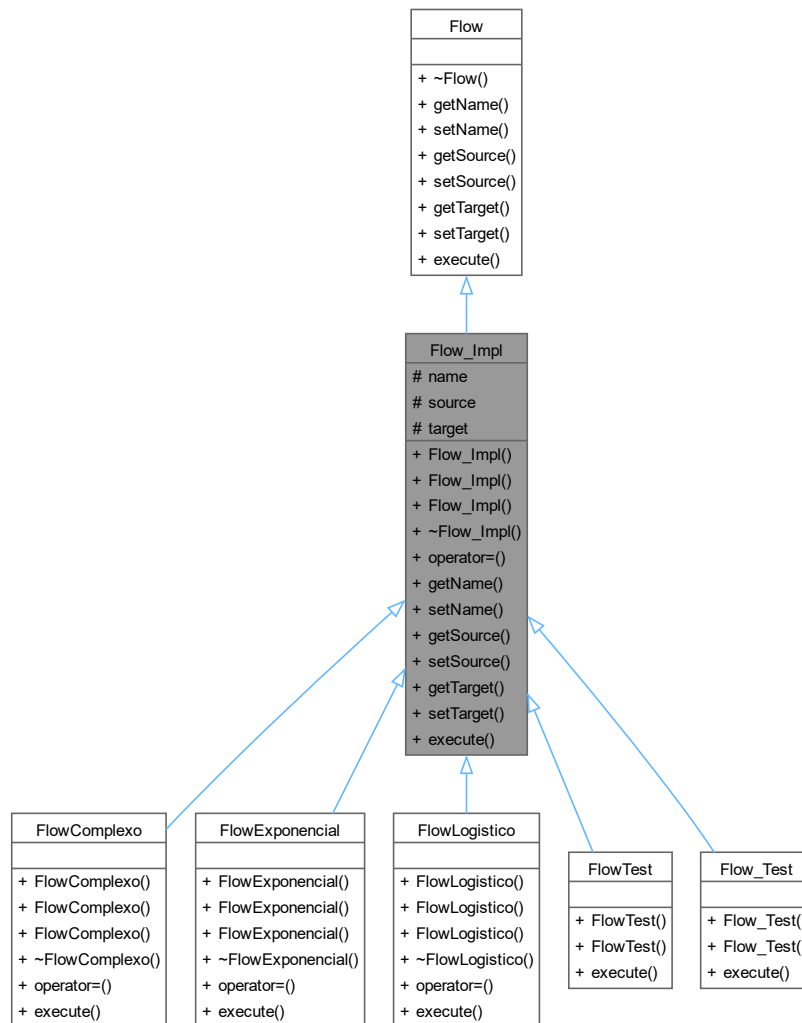
The documentation for this class was generated from the following file:

- include/[flow.h](#)

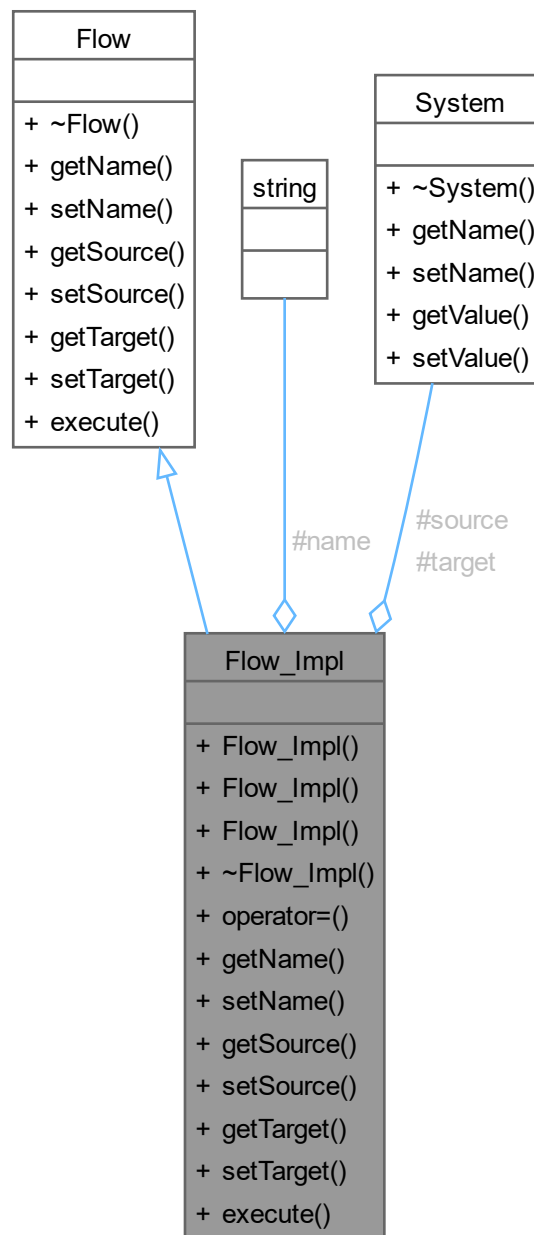
7.2 Flow_Impl Class Reference

```
#include <flow_impl.h>
```

Inheritance diagram for Flow_Impl:



Collaboration diagram for Flow_Impl:



Public Member Functions

- [Flow_Impl \(\)](#)
Construtor padrão da classe [Flow_Impl](#).
- [Flow_Impl \(const string &name, System *source, System *target\)](#)
Construtor parametrizado da classe [Flow](#).
- [Flow_Impl \(const Flow_Impl &other\)](#)
Construtor de cópia da classe [Flow](#).
- virtual [~Flow_Impl \(\)](#)

- Destruitor da classe [Flow](#).
- [Flow_Impl](#) & [operator=](#) (const [Flow_Impl](#) &other)
Operador de atribuição da classe [Flow](#).
- string [getName](#) () const
Retorna o nome do fluxo.
- void [setName](#) (const string &name)
Define o nome do fluxo.
- [System](#) * [getSource](#) () const
Retorna o sistema de origem.
- void [setSource](#) ([System](#) *source)
Define o sistema de origem.
- [System](#) * [getTarget](#) () const
Retorna o sistema de destino.
- void [setTarget](#) ([System](#) *target)
Define o sistema de destino.
- virtual double [execute](#) ()=0
Executa o cálculo do fluxo.

Public Member Functions inherited from [Flow](#)

- virtual [~Flow](#) ()=default
Destruitor virtual da classe [Flow](#).

Protected Attributes

- string [name](#)
- [System](#) * [source](#)
- [System](#) * [target](#)

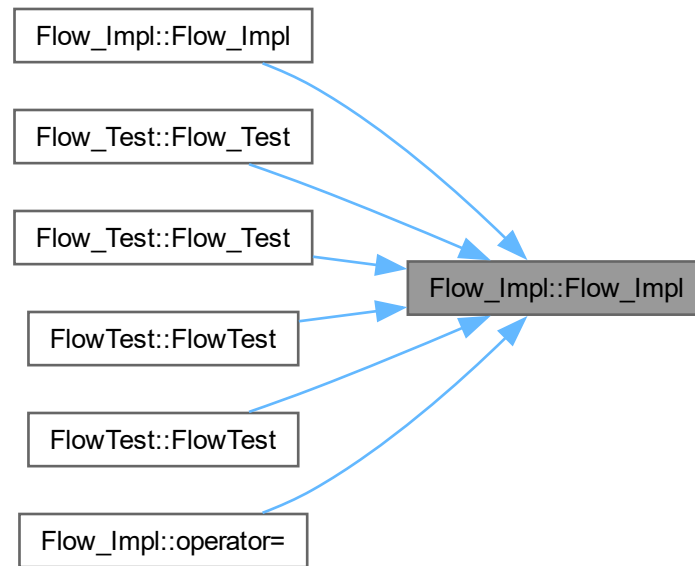
7.2.1 Constructor & Destructor Documentation

7.2.1.1 [Flow_Impl](#)() [1/3]

[Flow_Impl](#)::[Flow_Impl](#) ()

Construtor padrão da classe [Flow_Impl](#).

Inicializa o fluxo com nome vazio, source nulo e target nulo. Here is the caller graph for this function:



7.2.1.2 Flow_Impl() [2/3]

```
Flow_Impl::Flow_Impl (
    const string & name,
    System * source,
    System * target)
```

Construtor parametrizado da classe [Flow](#).

Construtor parametrizado da classe [Flow_Impl](#).

Parameters

name	Nome do fluxo.
source	Sistema de origem.
target	Sistema de destino.
name	Nome do fluxo.
source	Ponteiro para o sistema de origem.
target	Ponteiro para o sistema de destino.

7.2.1.3 Flow_Impl() [3/3]

```
Flow_Impl::Flow_Impl (
    const Flow_Impl & other)
```

Construtor de cópia da classe [Flow](#).

Construtor de cópia da classe [Flow_Impl](#).

Parameters

other	Outro objeto Flow .
other	Objeto Flow_Impl a ser copiado.

Here is the call graph for this function:



7.2.1.4 ~Flow_Impl()

Flow_Impl::~~Flow_Impl () [virtual]

Destrutor da classe [Flow](#).

Destrutor da classe [Flow_Impl](#).

7.2.2 Member Function Documentation

7.2.2.1 execute()

virtual double Flow_Impl::execute () [pure virtual]

Executa o cálculo do fluxo.

Método abstrato implementado pelas subclasses.

Returns

Valor calculado pelo fluxo.

Implements [Flow](#).

Implemented in [Flow_Test](#), [FlowComplexo](#), [FlowExponencial](#), [FlowLogistico](#), and [FlowTest](#).

7.2.2.2 getName()

string Flow_Impl::getName () const [virtual]

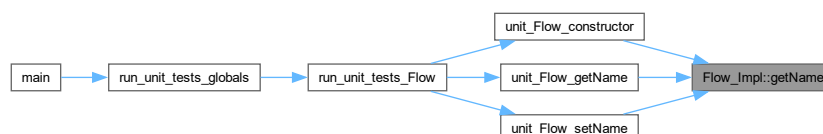
Retorna o nome do fluxo.

Returns

Nome do fluxo.

Implements [Flow](#).

Here is the caller graph for this function:



7.2.2.3 getSource()

`System * Flow_Impl::getSource () const [virtual]`

Retorna o sistema de origem.

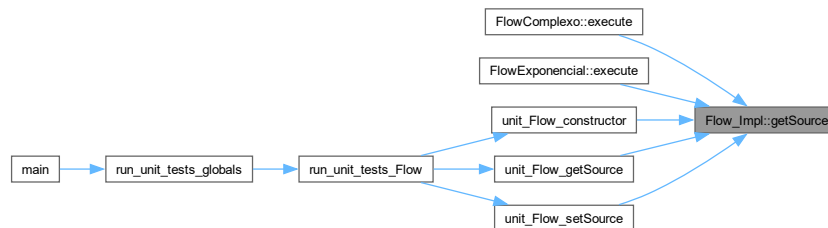
Retorna o ponteiro para o sistema de origem.

Returns

Ponteiro para o sistema de origem.

Implements [Flow](#).

Here is the caller graph for this function:



7.2.2.4 getTarget()

`System * Flow_Impl::getTarget () const [virtual]`

Retorna o sistema de destino.

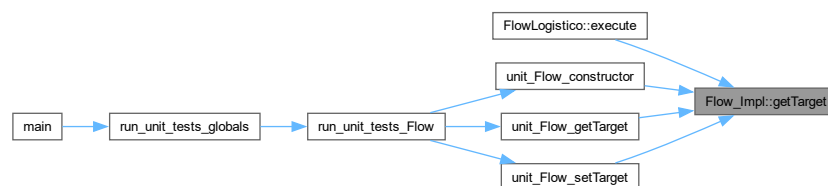
Retorna o ponteiro para o sistema de destino.

Returns

Ponteiro para o sistema de destino.

Implements [Flow](#).

Here is the caller graph for this function:



7.2.2.5 operator=()

`Flow_Impl & Flow_Impl::operator= (`
`const Flow_Impl & other)`

Operador de atribuição da classe [Flow](#).

Operador de atribuição da classe [Flow_Impl](#).

Copia os atributos de outro fluxo.

Parameters

other	Outro objeto Flow .
-------	-------------------------------------

Returns

Referência para o objeto atual.

Copia nome, source e target de outro objeto [Flow_Impl](#).

Parameters

other	Objeto Flow_Impl a ser atribuído.
-------	---

Returns

Referência para o objeto atual.

Here is the call graph for this function:



7.2.2.6 setName()

```
void Flow_Impl::setName (
    const string & name) [virtual]
```

Define o nome do fluxo.

Parameters

name	Novo nome do fluxo.
------	---------------------

Implements [Flow](#).

Here is the caller graph for this function:



7.2.2.7 setSource()

```
void Flow_Impl::setSource (
    System * source) [virtual]
```

Define o sistema de origem.

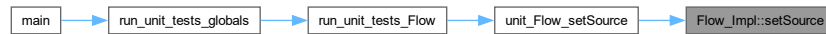
Define o sistema de origem do fluxo.

Parameters

source	Novo sistema de origem.
source	Ponteiro para o novo sistema de origem.

Implements [Flow](#).

Here is the caller graph for this function:



7.2.2.8 setTarget()

```
void Flow_Impl::setTarget (
    System * target) [virtual]
```

Define o sistema de destino.

Define o sistema de destino do fluxo.

Parameters

target	Novo sistema de destino.
target	Ponteiro para o novo sistema de destino.

Implements [Flow](#).

Here is the caller graph for this function:



7.2.3 Field Documentation

7.2.3.1 name

```
string Flow_Impl::name [protected]
```

7.2.3.2 source

```
System* Flow_Impl::source [protected]
```

7.2.3.3 target

```
System* Flow_Impl::target [protected]
```

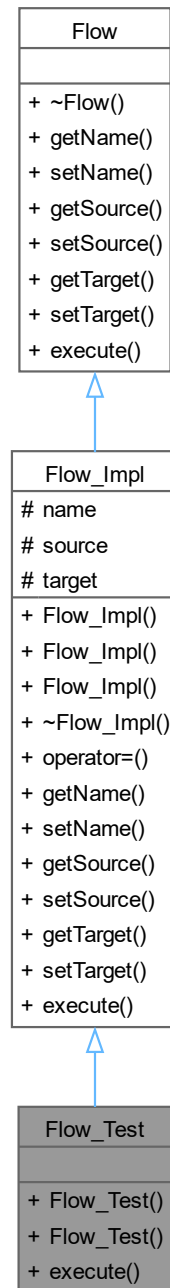
The documentation for this class was generated from the following files:

- include/[flow_impl.h](#)
- src/[flow_impl.cpp](#)

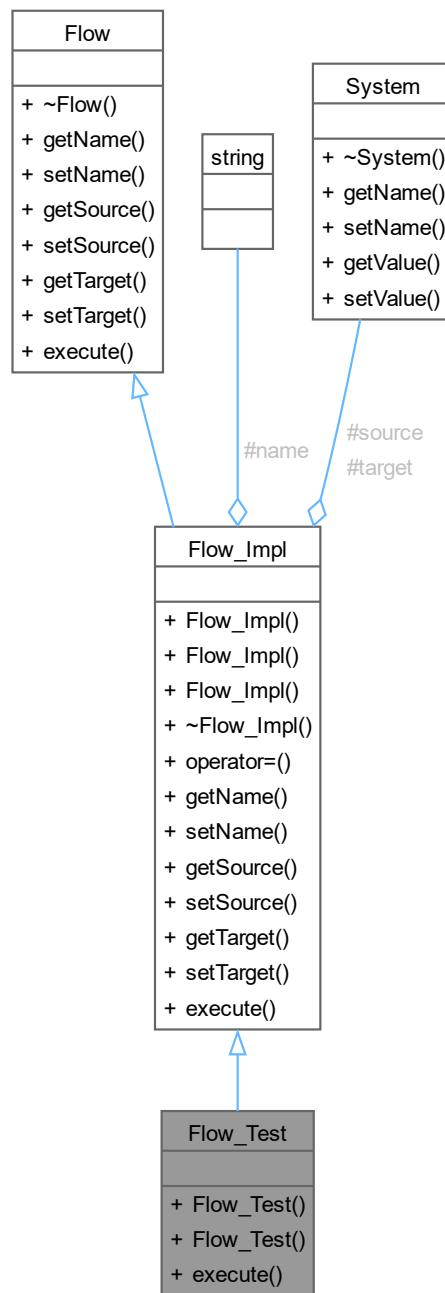
7.3 Flow_Test Class Reference

Implementação concreta de [Flow_Impl](#) utilizada nos testes unitários.

Inheritance diagram for Flow_Test:



Collaboration diagram for Flow_Test:



Public Member Functions

- [Flow_Test](#) ()
Construtor padrão de [Flow_Test](#).
- [Flow_Test](#) (string name, [System](#) *source, [System](#) *target)
Construtor parametrizado de [Flow_Test](#).
- double [execute](#) () override
Implementação trivial de [execute\(\)](#) para fins de teste.

Public Member Functions inherited from [Flow_Impl](#)

- [Flow_Impl](#) ()
Construtor padrão da classe [Flow_Impl](#).
- [Flow_Impl](#) (const string &name, [System](#) *source, [System](#) *target)
Construtor parametrizado da classe [Flow](#).
- [Flow_Impl](#) (const [Flow_Impl](#) &other)
Construtor de cópia da classe [Flow](#).
- virtual [~Flow_Impl](#) ()
Destrutor da classe [Flow](#).
- [Flow_Impl](#) & operator= (const [Flow_Impl](#) &other)
Operador de atribuição da classe [Flow](#).
- string [getName](#) () const
Retorna o nome do fluxo.
- void [setName](#) (const string &name)
Define o nome do fluxo.
- [System](#) * [getSource](#) () const
Retorna o sistema de origem.
- void [setSource](#) ([System](#) *source)
Define o sistema de origem.
- [System](#) * [getTarget](#) () const
Retorna o sistema de destino.
- void [setTarget](#) ([System](#) *target)
Define o sistema de destino.

Public Member Functions inherited from [Flow](#)

- virtual [~Flow](#) ()=default
Destrutor virtual da classe [Flow](#).

Additional Inherited Members

Protected Attributes inherited from [Flow_Impl](#)

- string [name](#)
- [System](#) * [source](#)
- [System](#) * [target](#)

7.3.1 Detailed Description

Implementação concreta de [Flow_Impl](#) utilizada nos testes unitários.

Classe auxiliar de teste que herda de [Flow_Impl](#) e fornece uma implementação trivial de [execute\(\)](#), retornando sempre 1.0. Permite testar os métodos de [Flow_Impl](#) sem depender de uma equação de fluxo real.

7.3.2 Constructor & Destructor Documentation

7.3.2.1 [Flow_Test](#)() [1/2]

[Flow_Test::Flow_Test](#) () [inline]

Construtor padrão de [Flow_Test](#).

Here is the call graph for this function:



7.3.2.2 Flow_Test() [2/2]

```
Flow_Test::Flow_Test (
    string name,
    System * source,
    System * target) [inline]
```

Construtor parametrizado de [Flow_Test](#).

Parameters

name	Nome do fluxo.
source	Ponteiro para o sistema de origem.
target	Ponteiro para o sistema de destino.

Here is the call graph for this function:



7.3.3 Member Function Documentation

7.3.3.1 execute()

```
double Flow_Test::execute () [inline], [override], [virtual]
```

Implementação trivial de [execute\(\)](#) para fins de teste.

Returns

Sempre retorna 1.0.

Implements [Flow_Impl](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

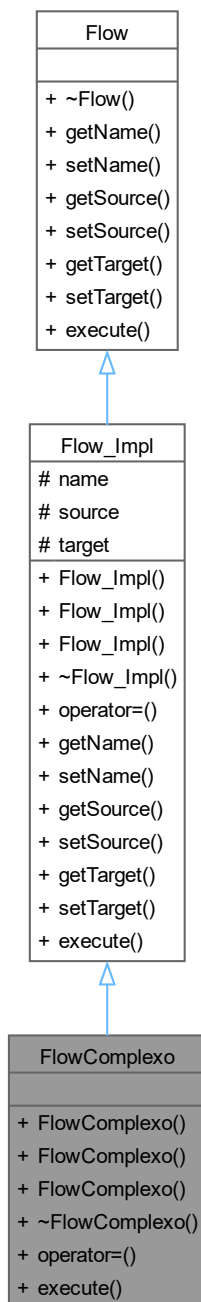
- test/unit/[unit_flow.cpp](#)

7.4 FlowComplexo Class Reference

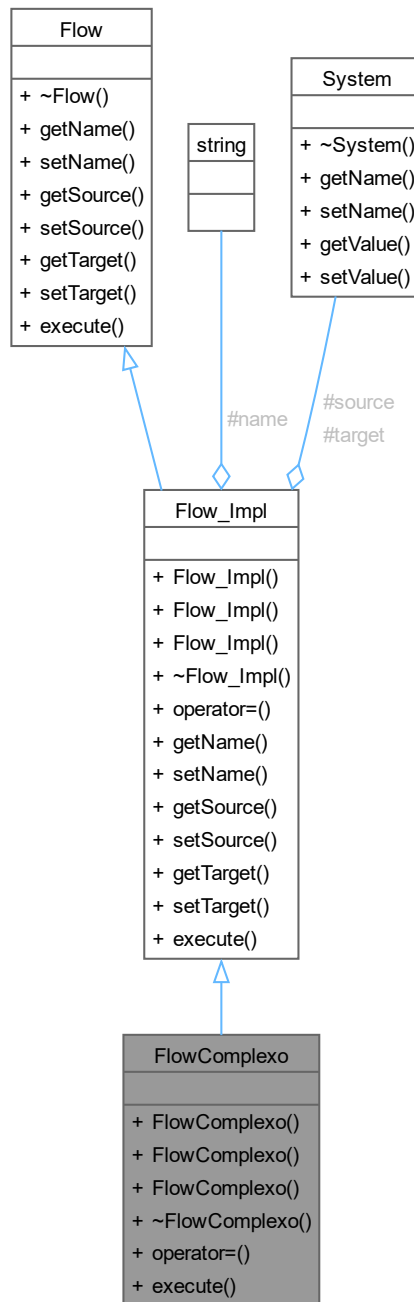
Fluxo com equação de múltiplas interações entre sistemas.

```
#include <flows.h>
```

Inheritance diagram for FlowComplexo:



Collaboration diagram for FlowComplexo:



Public Member Functions

- **FlowComplexo** ()
Construtor padrão da classe **FlowComplexo**.
- **FlowComplexo** (const string &name, **System** *source, **System** *target)
Construtor parametrizado da classe **FlowComplexo**.
- **FlowComplexo** (const **FlowComplexo** &other)
Construtor de cópia da classe **FlowComplexo**.

- virtual `~FlowComplexo ()`
Destrutor virtual da classe `FlowComplexo`.
- `FlowComplexo & operator= (const FlowComplexo &other)`
Operador de atribuição da classe `FlowComplexo`.
- double `execute ()` override
Executa o cálculo do fluxo complexo.

Public Member Functions inherited from `Flow_Impl`

- `Flow_Impl ()`
Construtor padrão da classe `Flow_Impl`.
- `Flow_Impl (const string &name, System *source, System *target)`
Construtor parametrizado da classe `Flow`.
- `Flow_Impl (const Flow_Impl &other)`
Construtor de cópia da classe `Flow`.
- virtual `~Flow_Impl ()`
Destrutor da classe `Flow`.
- `Flow_Impl & operator= (const Flow_Impl &other)`
Operador de atribuição da classe `Flow`.
- string `getName ()` const
Retorna o nome do fluxo.
- void `setName (const string &name)`
Define o nome do fluxo.
- `System * getSource ()` const
Retorna o sistema de origem.
- void `setSource (System *source)`
Define o sistema de origem.
- `System * getTarget ()` const
Retorna o sistema de destino.
- void `setTarget (System *target)`
Define o sistema de destino.

Public Member Functions inherited from `Flow`

- virtual `~Flow ()=default`
Destrutor virtual da classe `Flow`.

Additional Inherited Members

Protected Attributes inherited from `Flow_Impl`

- string `name`
- `System * source`
- `System * target`

7.4.1 Detailed Description

Fluxo com equação de múltiplas interações entre sistemas.

Implementa um fluxo mais elaborado que combina interações entre o sistema de origem e o sistema de destino, permitindo modelar dinâmicas com dependências cruzadas entre compartimentos, como modelos predador-presa ou transferências bidirecionalmente influenciadas.

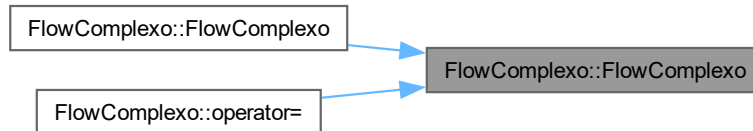
7.4.2 Constructor & Destructor Documentation

7.4.2.1 FlowComplexo() [1/3]

FlowComplexo::FlowComplexo ()

Construtor padrão da classe [FlowComplexo](#).

Here is the caller graph for this function:



7.4.2.2 FlowComplexo() [2/3]

FlowComplexo::FlowComplexo (
 const string & name,
 [System](#) * source,
 [System](#) * target)

Construtor parametrizado da classe [FlowComplexo](#).

Parameters

name	Nome do fluxo.
source	Ponteiro para o sistema de origem.
target	Ponteiro para o sistema de destino.
name	Nome do fluxo.
source	Sistema de origem.
target	Sistema de destino.

7.4.2.3 FlowComplexo() [3/3]

FlowComplexo::FlowComplexo (
 const [FlowComplexo](#) & other)

Construtor de cópia da classe [FlowComplexo](#).

Parameters

other	Objeto FlowComplexo a ser copiado.
other	Outro objeto FlowComplexo .

Here is the call graph for this function:



7.4.2.4 ~FlowComplexo()

FlowComplexo::~~FlowComplexo () [virtual]

Destrutor virtual da classe [FlowComplexo](#).

Destrutor da classe [FlowComplexo](#).

7.4.3 Member Function Documentation

7.4.3.1 execute()

double FlowComplexo::execute () [override], [virtual]

Executa o cálculo do fluxo complexo.

Calcula a quantidade transferida entre os sistemas com base em uma equação que combina os valores do sistema de origem e do sistema de destino, modelando interações mais elaboradas entre os compartimentos do modelo.

Returns

Valor calculado pelo fluxo complexo.

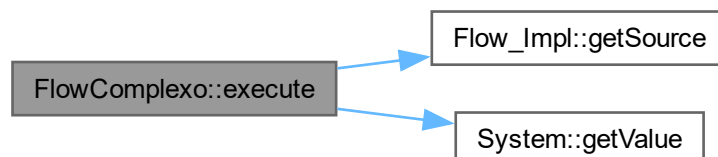
Calcula a transferência entre sistemas utilizando o modelo complexo.

Returns

Valor calculado do fluxo.

Implements [Flow_Impl](#).

Here is the call graph for this function:



7.4.3.2 operator=()

[FlowComplexo](#) & FlowComplexo::operator= (
const [FlowComplexo](#) & other)

Operador de atribuição da classe [FlowComplexo](#).

Parameters

other	Objeto FlowComplexo a ser atribuído.
-------	--

Returns

Referência para o objeto atual.

Parameters

other	Outro objeto FlowComplexo .
-------	---

Returns

Referência para o objeto atual.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

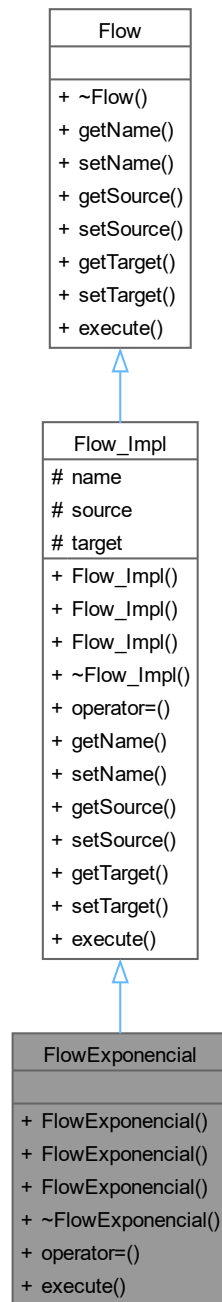
- include/[flows.h](#)
- test/functional/[flows.cpp](#)

7.5 FlowExponencial Class Reference

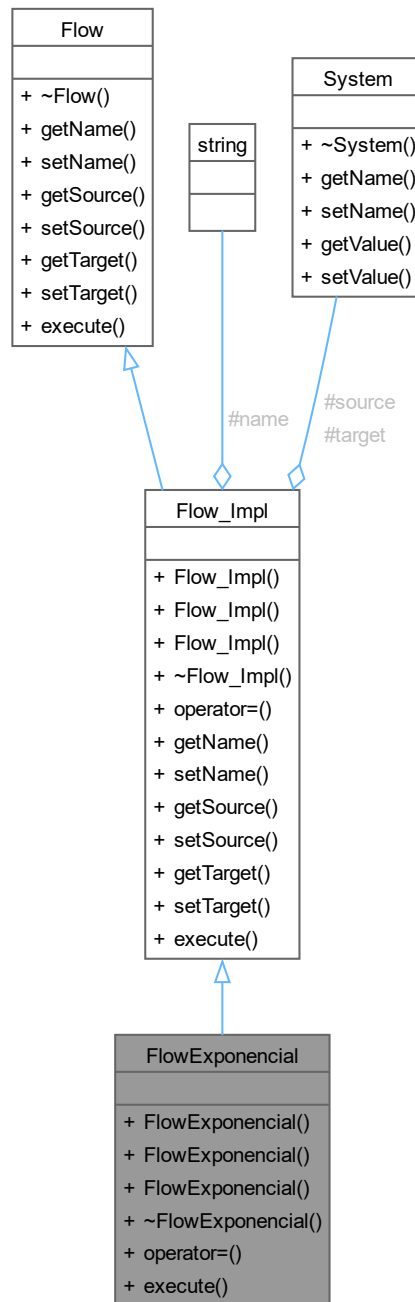
Fluxo baseado em crescimento exponencial.

```
#include <flows.h>
```

Inheritance diagram for FlowExponential:



Collaboration diagram for FlowExponencial:



Public Member Functions

- [FlowExponencial](#) ()
Construtor padrão da classe [FlowExponencial](#).
- [FlowExponencial](#) (const string &name, [System](#) *source, [System](#) *target)
Construtor parametrizado da classe [FlowExponencial](#).
- [FlowExponencial](#) (const [FlowExponencial](#) &other)
Construtor de cópia da classe [FlowExponencial](#).

- virtual `~FlowExponencial ()`
Destrutor virtual da classe `FlowExponencial`.
- `FlowExponencial & operator= (const FlowExponencial &other)`
Operador de atribuição da classe `FlowExponencial`.
- double `execute ()` override
Executa o cálculo do fluxo exponencial.

Public Member Functions inherited from `Flow_Impl`

- `Flow_Impl ()`
Construtor padrão da classe `Flow_Impl`.
- `Flow_Impl (const string &name, System *source, System *target)`
Construtor parametrizado da classe `Flow`.
- `Flow_Impl (const Flow_Impl &other)`
Construtor de cópia da classe `Flow`.
- virtual `~Flow_Impl ()`
Destrutor da classe `Flow`.
- `Flow_Impl & operator= (const Flow_Impl &other)`
Operador de atribuição da classe `Flow`.
- string `getName ()` const
Retorna o nome do fluxo.
- void `setName (const string &name)`
Define o nome do fluxo.
- `System * getSource ()` const
Retorna o sistema de origem.
- void `setSource (System *source)`
Define o sistema de origem.
- `System * getTarget ()` const
Retorna o sistema de destino.
- void `setTarget (System *target)`
Define o sistema de destino.

Public Member Functions inherited from `Flow`

- virtual `~Flow ()=default`
Destrutor virtual da classe `Flow`.

Additional Inherited Members

Protected Attributes inherited from `Flow_Impl`

- string `name`
- `System * source`
- `System * target`

7.5.1 Detailed Description

Fluxo baseado em crescimento exponencial.

Implementa uma equação de fluxo exponencial entre dois sistemas, onde a taxa de transferência é proporcional ao valor atual do sistema de origem. Tipicamente usada para modelar crescimento ou decaimento exponencial em populações e recursos.

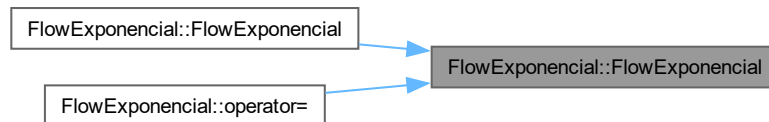
7.5.2 Constructor & Destructor Documentation

7.5.2.1 FlowExponencial() [1/3]

FlowExponencial::FlowExponencial ()

Construtor padrão da classe [FlowExponencial](#).

Here is the caller graph for this function:



7.5.2.2 FlowExponencial() [2/3]

FlowExponencial::FlowExponencial (
 const string & name,
 [System](#) * source,
 [System](#) * target)

Construtor parametrizado da classe [FlowExponencial](#).

Parameters

name	Nome do fluxo.
source	Ponteiro para o sistema de origem.
target	Ponteiro para o sistema de destino.
name	Nome do fluxo.
source	Sistema de origem.
target	Sistema de destino.

7.5.2.3 FlowExponencial() [3/3]

FlowExponencial::FlowExponencial (
 const [FlowExponencial](#) & other)

Construtor de cópia da classe [FlowExponencial](#).

Parameters

other	Objeto FlowExponencial a ser copiado.
other	Outro objeto FlowExponencial .

Here is the call graph for this function:



7.5.2.4 ~FlowExponencial()

FlowExponencial::~~FlowExponencial () [virtual]

Destrutor virtual da classe [FlowExponencial](#).

Destrutor da classe [FlowExponencial](#).

7.5.3 Member Function Documentation

7.5.3.1 execute()

double FlowExponencial::execute () [override], [virtual]

Executa o cálculo do fluxo exponencial.

Calcula a quantidade transferida do sistema de origem para o sistema de destino com base em uma equação de crescimento exponencial aplicada ao valor atual do sistema de origem.

Returns

Valor calculado pelo fluxo exponencial.

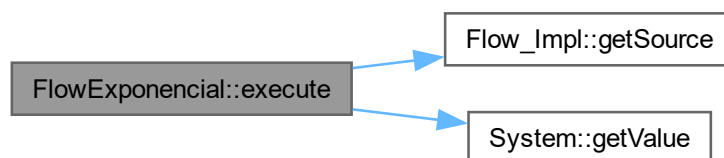
O valor transferido corresponde a 1% do valor do sistema de origem.

Returns

Valor calculado do fluxo.

Implements [Flow_Impl](#).

Here is the call graph for this function:



7.5.3.2 operator=()

[FlowExponencial](#) & FlowExponencial::operator= (

const [FlowExponencial](#) & other)

Operador de atribuição da classe [FlowExponencial](#).

Parameters

other	Objeto FlowExponencial a ser atribuído.
-------	---

Returns

Referência para o objeto atual.

Parameters

other	Outro objeto FlowExponencial .
-------	--

Returns

Referência para o objeto atual.

Here is the call graph for this function:



The documentation for this class was generated from the following files:

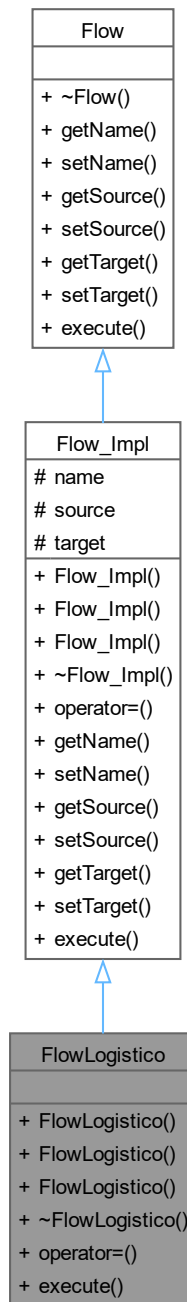
- include/[flows.h](#)
- test/functional/[flows.cpp](#)

7.6 FlowLogistico Class Reference

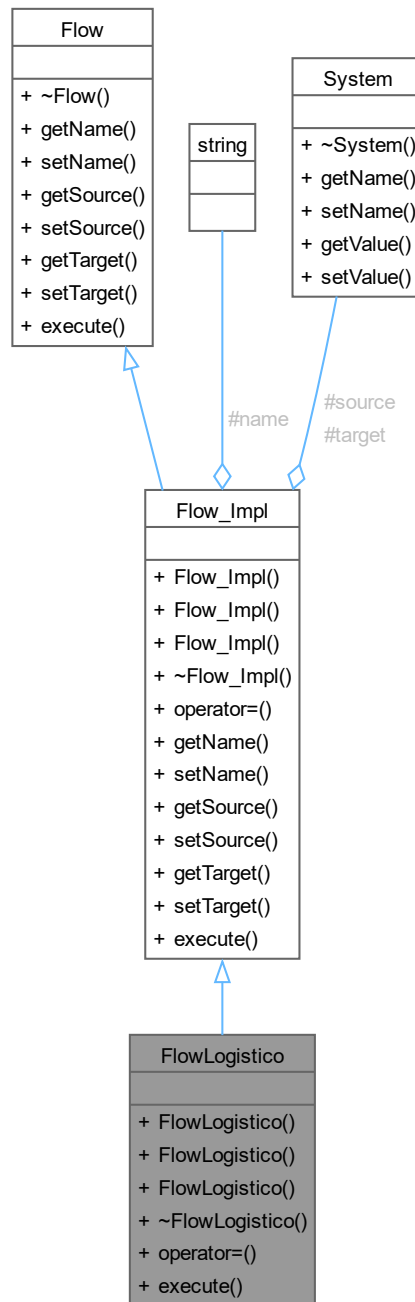
Fluxo baseado em crescimento logístico.

```
#include <flows.h>
```

Inheritance diagram for FlowLogistico:



Collaboration diagram for FlowLogistico:



Public Member Functions

- [FlowLogistico](#) ()
Construtor padrão da classe [FlowLogistico](#).
- [FlowLogistico](#) (const string &name, [System](#) *source, [System](#) *target)
Construtor parametrizado da classe [FlowLogistico](#).
- [FlowLogistico](#) (const [FlowLogistico](#) &other)
Construtor de cópia da classe [FlowLogistico](#).

- virtual `~FlowLogistico ()`
Destrutor virtual da classe `FlowLogistico`.
- `FlowLogistico & operator= (const FlowLogistico &other)`
Operador de atribuição da classe `FlowLogistico`.
- double `execute ()` override
Executa o cálculo do fluxo logístico.

Public Member Functions inherited from `Flow_Impl`

- `Flow_Impl ()`
Construtor padrão da classe `Flow_Impl`.
- `Flow_Impl (const string &name, System *source, System *target)`
Construtor parametrizado da classe `Flow`.
- `Flow_Impl (const Flow_Impl &other)`
Construtor de cópia da classe `Flow`.
- virtual `~Flow_Impl ()`
Destrutor da classe `Flow`.
- `Flow_Impl & operator= (const Flow_Impl &other)`
Operador de atribuição da classe `Flow`.
- string `getName ()` const
Retorna o nome do fluxo.
- void `setName (const string &name)`
Define o nome do fluxo.
- `System * getSource ()` const
Retorna o sistema de origem.
- void `setSource (System *source)`
Define o sistema de origem.
- `System * getTarget ()` const
Retorna o sistema de destino.
- void `setTarget (System *target)`
Define o sistema de destino.

Public Member Functions inherited from `Flow`

- virtual `~Flow ()=default`
Destrutor virtual da classe `Flow`.

Additional Inherited Members

Protected Attributes inherited from `Flow_Impl`

- string `name`
- `System * source`
- `System * target`

7.6.1 Detailed Description

Fluxo baseado em crescimento logístico.

Implementa uma equação de fluxo logístico entre dois sistemas, onde a taxa de transferência desacelera conforme o sistema de origem se aproxima de uma capacidade limite. Usada para modelar crescimento populacional com capacidade de suporte.

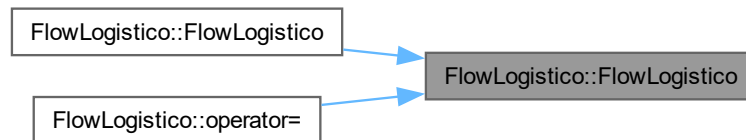
7.6.2 Constructor & Destructor Documentation

7.6.2.1 FlowLogistico() [1/3]

FlowLogistico::FlowLogistico ()

Construtor padrão da classe [FlowLogistico](#).

Here is the caller graph for this function:



7.6.2.2 FlowLogistico() [2/3]

FlowLogistico::FlowLogistico (
 const string & name,
 [System](#) * source,
 [System](#) * target)

Construtor parametrizado da classe [FlowLogistico](#).

Parameters

name	Nome do fluxo.
source	Ponteiro para o sistema de origem.
target	Ponteiro para o sistema de destino.
name	Nome do fluxo.
source	Sistema de origem.
target	Sistema de destino.

7.6.2.3 FlowLogistico() [3/3]

FlowLogistico::FlowLogistico (
 const [FlowLogistico](#) & other)

Construtor de cópia da classe [FlowLogistico](#).

Parameters

other	Objeto FlowLogistico a ser copiado.
other	Outro objeto FlowLogistico .

Here is the call graph for this function:



7.6.2.4 ~FlowLogistico()

FlowLogistico::~~FlowLogistico () [virtual]

Destrutor virtual da classe [FlowLogistico](#).

Destrutor da classe [FlowLogistico](#).

7.6.3 Member Function Documentation

7.6.3.1 execute()

double FlowLogistico::execute () [override], [virtual]

Executa o cálculo do fluxo logístico.

Calcula a quantidade transferida do sistema de origem para o sistema de destino com base em uma equação de crescimento logístico, levando em conta os valores atuais de ambos os sistemas.

Returns

Valor calculado pelo fluxo logístico.

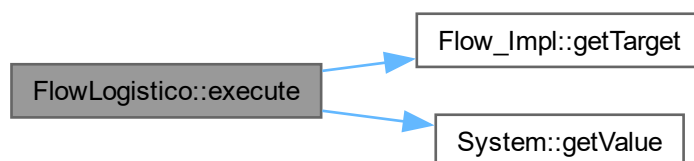
Calcula o crescimento logístico considerando a capacidade limite do sistema.

Returns

Valor calculado do fluxo.

Implements [Flow_Impl](#).

Here is the call graph for this function:



7.6.3.2 operator=()

[FlowLogistico](#) & FlowLogistico::operator= (
const [FlowLogistico](#) & other)

Operador de atribuição da classe [FlowLogistico](#).

Parameters

other	Objeto FlowLogistico a ser atribuído.
-------	---

Returns

Referência para o objeto atual.

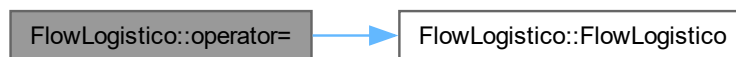
Parameters

other	Outro objeto FlowLogistico .
-------	--

Returns

Referência para o objeto atual.

Here is the call graph for this function:



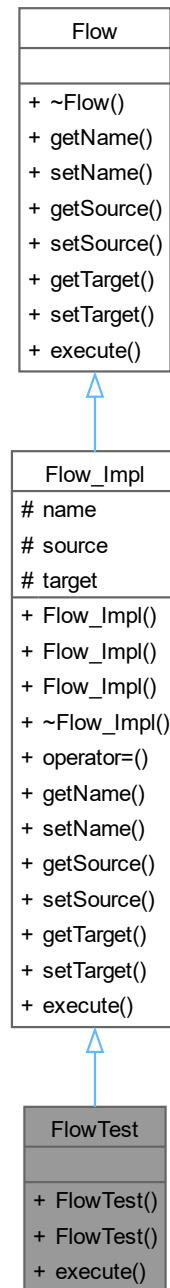
The documentation for this class was generated from the following files:

- include/[flows.h](#)
- test/functional/[flows.cpp](#)

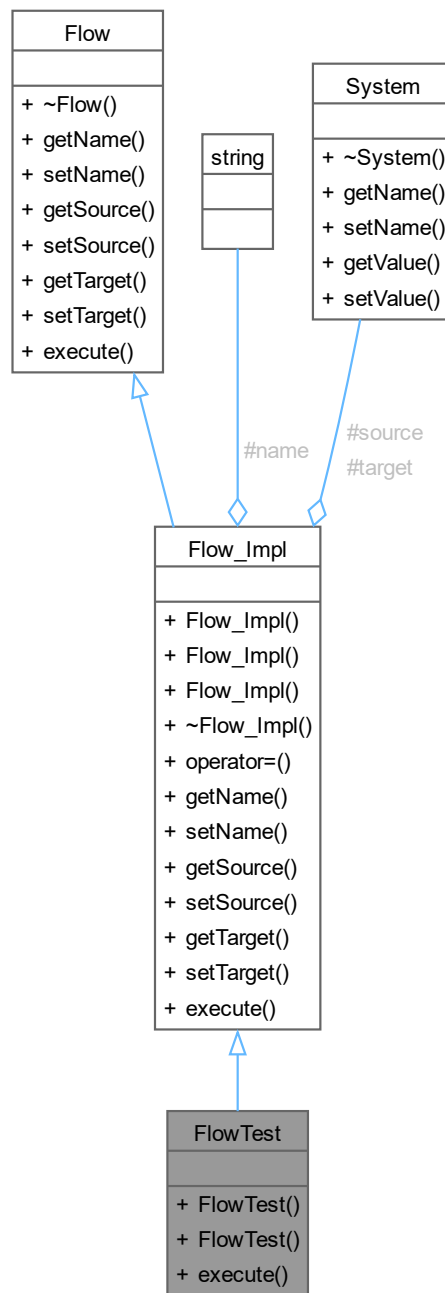
7.7 FlowTest Class Reference

Implementação concreta de [Flow_Impl](#) utilizada nos testes do [Model](#).

Inheritance diagram for FlowTest:



Collaboration diagram for FlowTest:



Public Member Functions

- [FlowTest](#) ()
Construtor padrão de [FlowTest](#).
- [FlowTest](#) (string name, [System](#) *source, [System](#) *target)
Construtor parametrizado de [FlowTest](#).
- double [execute](#) () override
Implementação trivial de [execute\(\)](#) para fins de teste.

Public Member Functions inherited from [Flow_Impl](#)

- [Flow_Impl](#) ()
Construtor padrão da classe [Flow_Impl](#).
- [Flow_Impl](#) (const string &name, [System](#) *source, [System](#) *target)
Construtor parametrizado da classe [Flow](#).
- [Flow_Impl](#) (const [Flow_Impl](#) &other)
Construtor de cópia da classe [Flow](#).
- virtual [~Flow_Impl](#) ()
Destrutor da classe [Flow](#).
- [Flow_Impl](#) & operator= (const [Flow_Impl](#) &other)
Operador de atribuição da classe [Flow](#).
- string [getName](#) () const
Retorna o nome do fluxo.
- void [setName](#) (const string &name)
Define o nome do fluxo.
- [System](#) * [getSource](#) () const
Retorna o sistema de origem.
- void [setSource](#) ([System](#) *source)
Define o sistema de origem.
- [System](#) * [getTarget](#) () const
Retorna o sistema de destino.
- void [setTarget](#) ([System](#) *target)
Define o sistema de destino.

Public Member Functions inherited from [Flow](#)

- virtual [~Flow](#) ()=default
Destrutor virtual da classe [Flow](#).

Additional Inherited Members

Protected Attributes inherited from [Flow_Impl](#)

- string [name](#)
- [System](#) * [source](#)
- [System](#) * [target](#)

7.7.1 Detailed Description

Implementação concreta de [Flow_Impl](#) utilizada nos testes do [Model](#).

Classe auxiliar de teste que herda de [Flow_Impl](#) e fornece uma implementação trivial de [execute\(\)](#), retornando sempre 1.0. Permite testar a lógica de execução do [Model](#) sem depender de uma equação de fluxo real.

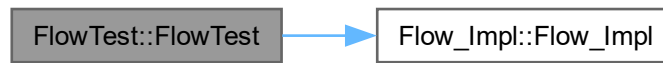
7.7.2 Constructor & Destructor Documentation

7.7.2.1 FlowTest() [1/2]

[FlowTest::FlowTest](#) () [inline]

Construtor padrão de [FlowTest](#).

Here is the call graph for this function:



7.7.2.2 FlowTest() [2/2]

```
FlowTest::FlowTest (
    string name,
    System * source,
    System * target) [inline]
```

Construtor parametrizado de [FlowTest](#).

Parameters

name	Nome do fluxo.
source	Ponteiro para o sistema de origem.
target	Ponteiro para o sistema de destino.

Here is the call graph for this function:



7.7.3 Member Function Documentation

7.7.3.1 execute()

```
double FlowTest::execute () [inline], [override], [virtual]
```

Implementação trivial de [execute\(\)](#) para fins de teste.

Returns

Sempre retorna 1.0.

Implements [Flow_Impl](#).

The documentation for this class was generated from the following file:

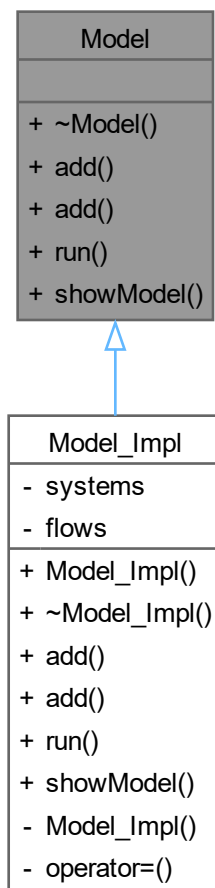
- test/unit/[unit_model.cpp](#)

7.8 Model Class Reference

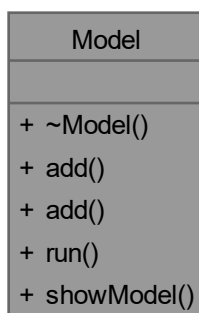
Interface abstrata que representa um modelo de dinâmica de sistemas.

```
#include <model.h>
```

Inheritance diagram for Model:



Collaboration diagram for Model:



Public Member Functions

- virtual `~Model()`=default
Destrutor virtual da classe `Model`.
- virtual void `add(System *s)`=0
Adiciona um sistema ao modelo.
- virtual void `add(Flow *f)`=0
Adiciona um fluxo ao modelo.
- virtual bool `run(int t_init, int t_final)`=0
Executa a simulação do modelo no intervalo de tempo especificado.
- virtual void `showModel()` const =0
Exibe no console as informações dos sistemas e fluxos do modelo.

7.8.1 Detailed Description

Interface abstrata que representa um modelo de dinâmica de sistemas.

A classe `Model` define a interface para um modelo de simulação composto por sistemas (estoques) e fluxos (equações de transferência). O modelo é responsável por gerenciar esses elementos e conduzir a simulação ao longo de um intervalo de tempo discreto. Subclasses devem fornecer implementação concreta para todos os métodos virtuais puros.

7.8.2 Constructor & Destructor Documentation

7.8.2.1 ~Model()

virtual `Model::~Model()` [virtual], [default]

Destrutor virtual da classe `Model`.

7.8.3 Member Function Documentation

7.8.3.1 add() [1/2]

virtual void `Model::add(Flow * f)` [pure virtual]

Adiciona um fluxo ao modelo.

Parameters

f	Ponteiro para o fluxo a ser adicionado.
---	---

Implemented in `Model_Impl`.

7.8.3.2 add() [2/2]

virtual void `Model::add(System * s)` [pure virtual]

Adiciona um sistema ao modelo.

Parameters

s	Ponteiro para o sistema a ser adicionado.
---	---

Implemented in `Model_Impl`.

7.8.3.3 run()

virtual bool `Model::run(int t_init, int t_final)` [pure virtual]

Executa a simulação do modelo no intervalo de tempo especificado.

A cada passo de tempo, todos os fluxos são calculados com base nos valores atuais dos sistemas, e os sistemas são atualizados em seguida.

Parameters

t_init	Tempo inicial da simulação (inclusive).
t_final	Tempo final da simulação (inclusive).

Returns

true se a execução foi concluída com sucesso; false caso contrário.

Implemented in [Model_Impl](#).

7.8.3.4 showModel()

virtual void Model::showModel () const [pure virtual]

Exibe no console as informações dos sistemas e fluxos do modelo.

Implemented in [Model_Impl](#).

The documentation for this class was generated from the following file:

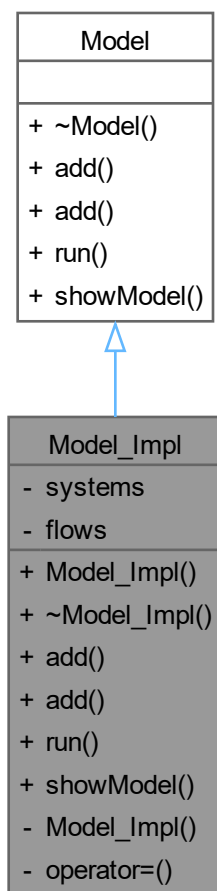
- include/[model.h](#)

7.9 Model_Impl Class Reference

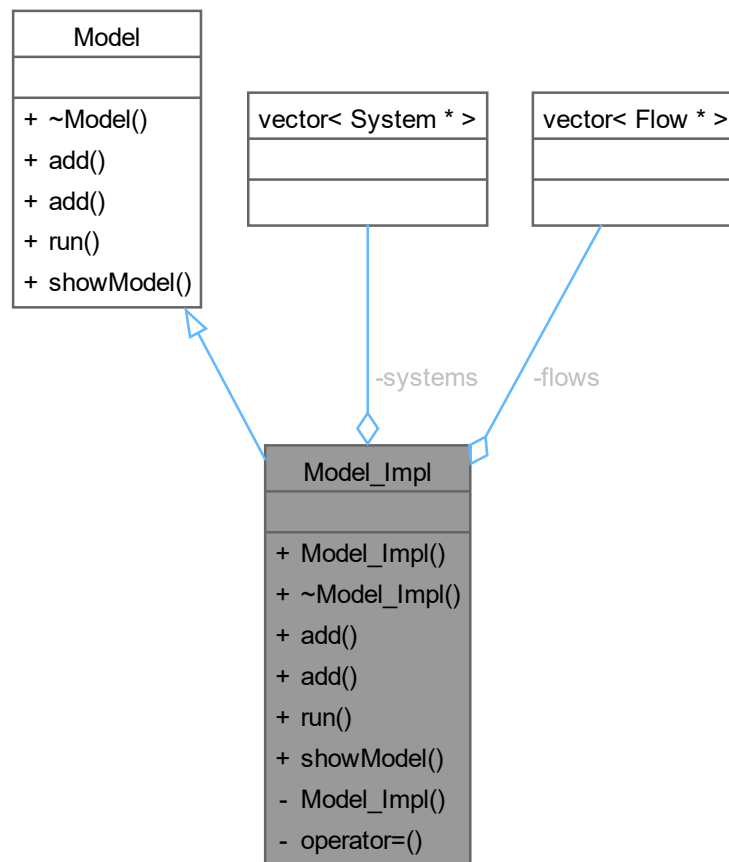
Classe responsável por representar um modelo de simulação.

#include <model_impl.h>

Inheritance diagram for Model_Impl:



Collaboration diagram for Model_Impl:



Public Member Functions

- [Model_Impl](#) ()
Construtor padrão da classe [Model](#).
- virtual [~Model_Impl](#) ()
Destrutor virtual da classe [Model](#).
- void [add](#) ([System](#) *s)
Adiciona um sistema ao modelo.
- void [add](#) ([Flow](#) *f)
Adiciona um fluxo ao modelo.
- bool [run](#) (int t_init, int t_final)
Executa a simulação do modelo.
- void [showModel](#) () const
Exibe informações do modelo.

Public Member Functions inherited from [Model](#)

- virtual [~Model](#) ()=default
Destrutor virtual da classe [Model](#).

Private Member Functions

- [Model_Impl](#) (const [Model_Impl](#) &other)
Construtor de cópia privado.
- [Model_Impl](#) & operator= (const [Model_Impl](#) &other)
Operador de atribuição privado.

Private Attributes

- vector< [System](#) * > [systems](#)
Vetor contendo os sistemas do modelo.
- vector< [Flow](#) * > [flows](#)
Vetor contendo os fluxos do modelo.

7.9.1 Detailed Description

Classe responsável por representar um modelo de simulação.

A classe [Model_Impl](#) armazena os sistemas e fluxos que compõem um modelo de dinâmica de sistemas, sendo responsável pela execução da simulação.

7.9.2 Constructor & Destructor Documentation

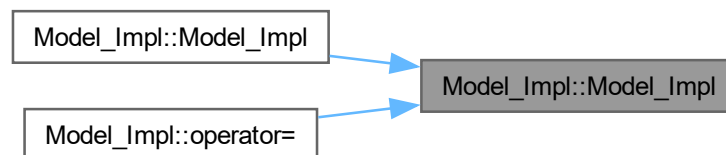
7.9.2.1 Model_Impl() [1/2]

[Model_Impl::Model_Impl](#) ()

Construtor padrão da classe [Model](#).

Construtor padrão da classe [Model_Impl](#).

Inicializa o modelo com vetores de sistemas e fluxos vazios. Here is the caller graph for this function:



7.9.2.2 ~Model_Impl()

[Model_Impl::~~Model_Impl](#) () [virtual]

Destrutor virtual da classe [Model](#).

Destrutor da classe [Model_Impl](#).

Limpa os vetores de sistemas e fluxos. Os objetos apontados pelos ponteiros não são destruídos (responsabilidade do código cliente).

7.9.2.3 Model_Impl() [2/2]

[Model_Impl::Model_Impl](#) (

const [Model_Impl](#) & other) [private]

Construtor de cópia privado.

Construtor de cópia da classe [Model_Impl](#).

Impede cópia da classe [Model](#).

Parameters

other	Outro objeto Model .
-------	--------------------------------------

Copia os vetores de sistemas e fluxos de outro modelo.

Parameters

other	Objeto Model_Impl a ser copiado.
-------	--

Here is the call graph for this function:



7.9.3 Member Function Documentation

7.9.3.1 add() [1/2]

void `Model_Impl::add` (
[Flow](#) * f) [virtual]

Adiciona um fluxo ao modelo.

Parameters

f	Ponteiro para o fluxo a ser adicionado.
---	---

Implements [Model](#).

7.9.3.2 add() [2/2]

void `Model_Impl::add` (
[System](#) * s) [virtual]

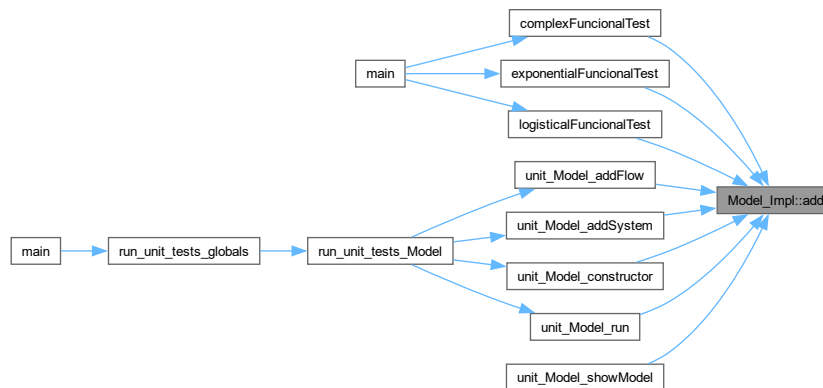
Adiciona um sistema ao modelo.

Parameters

s	Ponteiro para o sistema a ser adicionado.
---	---

Implements [Model](#).

Here is the caller graph for this function:



7.9.3.3 operator=()

`Model_Impl & Model_Impl::operator= (const Model_Impl & other) [private]`

Operador de atribuição privado.

Operador de atribuição da classe `Model_Impl`.

Impede atribuição entre objetos `Model`.

Parameters

other	Outro objeto <code>Model</code> .
-------	-----------------------------------

Returns

Referência para o próprio objeto.

Copia os vetores de sistemas e fluxos de outro modelo.

Parameters

other	Objeto <code>Model_Impl</code> a ser atribuído.
-------	---

Returns

Referência para o objeto atual.

Here is the call graph for this function:



7.9.3.4 run()

```
bool Model_Impl::run (  
    int t_init,  
    int t_final) [virtual]
```

Executa a simulação do modelo.

Executa a simulação do modelo no intervalo de tempo especificado.

Parameters

t_init	Tempo inicial da simulação.
t_final	Tempo final da simulação.

Returns

true caso a execução seja concluída com sucesso.

false caso ocorra falha na execução.

A cada passo de tempo, todos os fluxos são calculados com base nos valores atuais dos sistemas (passo síncrono), e em seguida os sistemas de origem e destino de cada fluxo são atualizados simultaneamente.

Parameters

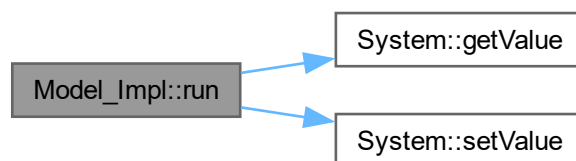
t_init	Tempo inicial da simulação (inclusive).
t_final	Tempo final da simulação (exclusive).

Returns

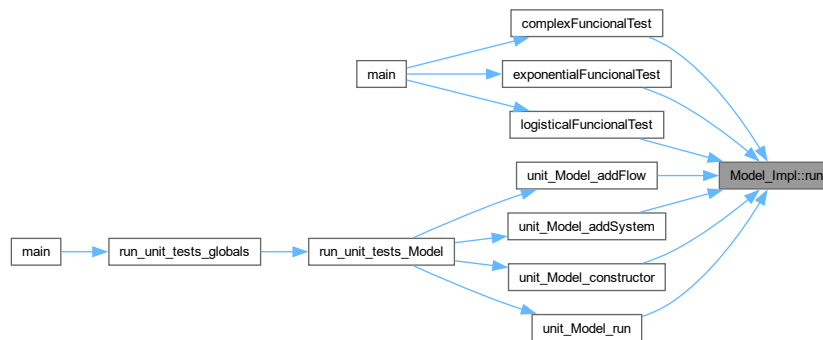
true se a execução foi concluída com sucesso.

Implements [Model](#).

Here is the call graph for this function:



Here is the caller graph for this function:



7.9.3.5 showModel()

```
void Model_Impl::showModel () const [virtual]
```

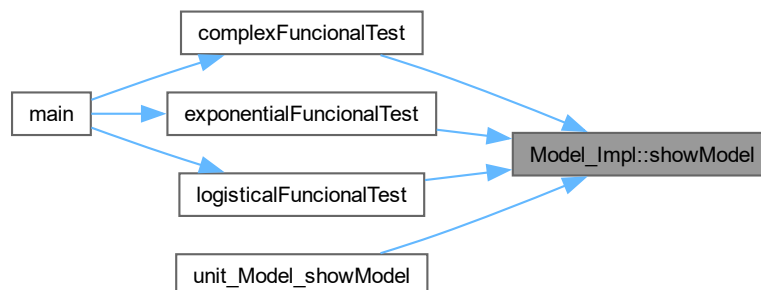
Exibe informações do modelo.

Exibe no console os sistemas e fluxos do modelo.

Para cada sistema, imprime seu nome e valor atual. Para cada fluxo, imprime a conexão entre sistema de origem e sistema de destino no formato "origem -> destino".

Implements [Model](#).

Here is the caller graph for this function:



7.9.4 Field Documentation

7.9.4.1 flows

```
vector<Flow*> Model_Impl::flows [private]
```

Vetor contendo os fluxos do modelo.

7.9.4.2 systems

```
vector<System*> Model_Impl::systems [private]
```

Vetor contendo os sistemas do modelo.

The documentation for this class was generated from the following files:

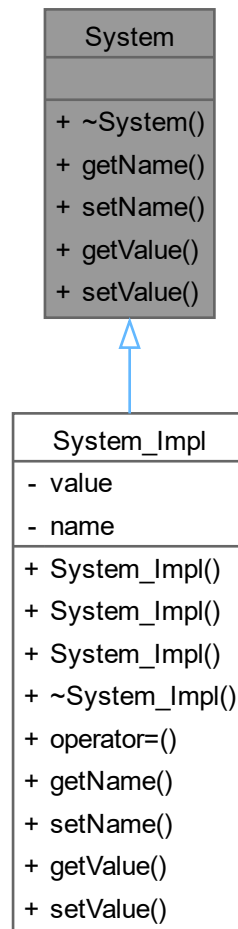
- include/[model_impl.h](#)
- src/[model_impl.cpp](#)

7.10 System Class Reference

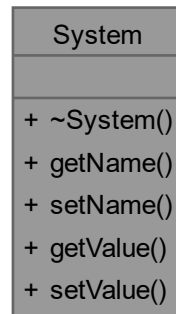
Interface abstrata que representa um sistema em um modelo de simulação.

#include <system.h>

Inheritance diagram for System:



Collaboration diagram for System:



Public Member Functions

- virtual [~System](#) ()=default
Destrutor virtual da classe [System](#).
- virtual string [getName](#) () const =0
Retorna o nome do sistema.
- virtual void [setName](#) (const string &name)=0
Define o nome do sistema.
- virtual double [getValue](#) () const =0
Retorna o valor atual armazenado no sistema.
- virtual void [setValue](#) (double value)=0
Define o valor armazenado no sistema.

7.10.1 Detailed Description

Interface abstrata que representa um sistema em um modelo de simulação.

A classe [System](#) define a interface para os compartimentos (estoques) de um modelo de dinâmica de sistemas. Um sistema armazena um valor numérico que é modificado ao longo do tempo pelos fluxos conectados a ele. Subclasses devem fornecer implementação concreta para todos os métodos virtuais puros.

7.10.2 Constructor & Destructor Documentation

7.10.2.1 ~System()

virtual System::~~System () [virtual], [default]

Destrutor virtual da classe [System](#).

7.10.3 Member Function Documentation

7.10.3.1 getName()

virtual string System::getName () const [pure virtual]

Retorna o nome do sistema.

Returns

Nome do sistema.

Implemented in [System_Impl](#).

7.10.3.2 getValue()

virtual double System::getValue () const [pure virtual]

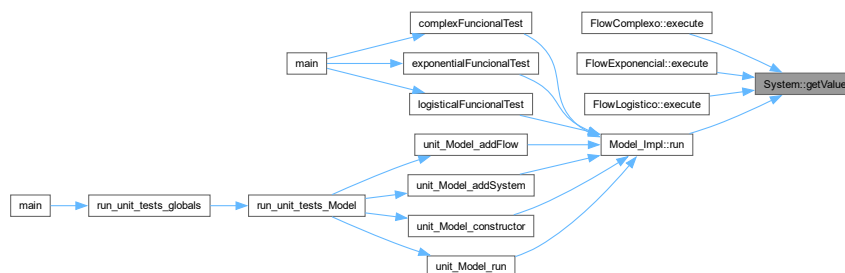
Retorna o valor atual armazenado no sistema.

Returns

Valor do sistema.

Implemented in [System_Impl](#).

Here is the caller graph for this function:



7.10.3.3 setName()

virtual void System::setName (const string & name) [pure virtual]

Define o nome do sistema.

Parameters

name	Novo nome do sistema.
------	-----------------------

Implemented in [System_Impl](#).

7.10.3.4 setValue()

virtual void System::setValue (double value) [pure virtual]

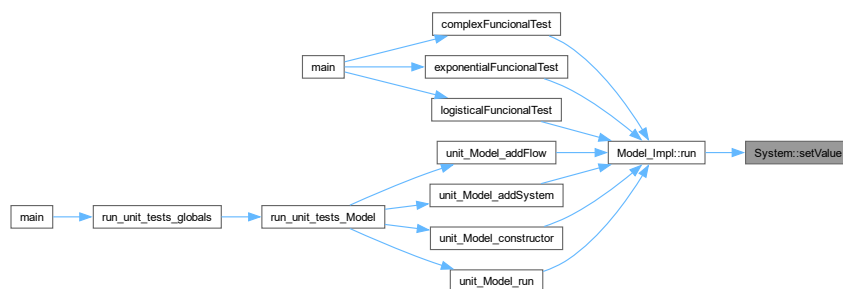
Define o valor armazenado no sistema.

Parameters

value	Novo valor do sistema.
-------	------------------------

Implemented in [System_Impl](#).

Here is the caller graph for this function:



The documentation for this class was generated from the following file:

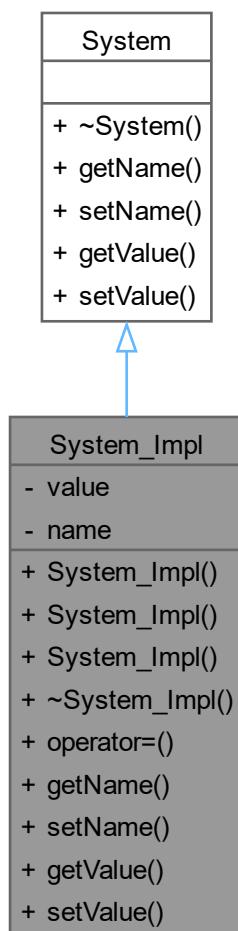
- `include/system.h`

7.11 System__Impl Class Reference

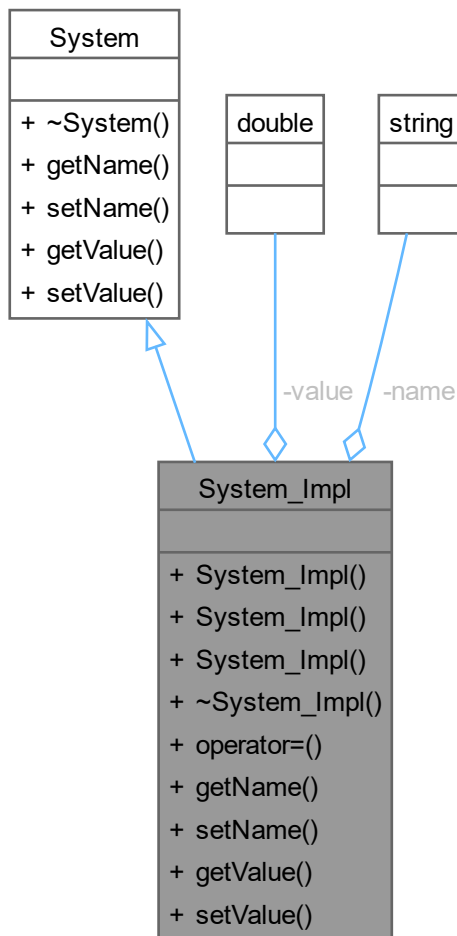
Classe responsável por representar um sistema.

```
#include <system_impl.h>
```

Inheritance diagram for System_Impl:



Collaboration diagram for `System_Impl`:



Public Member Functions

- [System_Impl \(\)](#)
Construtor padrão da classe [System](#).
- [System_Impl \(const string &name, double value\)](#)
Construtor da classe [System](#).
- [System_Impl \(const System_Impl &other\)](#)
Construtor de cópia privado.
- `virtual ~System_Impl ()`
Destrutor virtual da classe [System](#).
- [System_Impl & operator= \(const System_Impl &other\)](#)
Operador de atribuição privado.
- `string getName () const`
Retorna o nome do sistema.
- `void setName (const string &name)`
Define o nome do sistema.
- `double getValue () const`

- Retorna o valor atual do sistema.
- void [setValue](#) (double value)
Define o valor do sistema.

Public Member Functions inherited from [System](#)

- virtual [~System](#) ()=default
Destrutor virtual da classe [System](#).

Private Attributes

- double [value](#)
Valor armazenado pelo sistema.
- string [name](#)
Nome do sistema.

7.11.1 Detailed Description

Classe responsável por representar um sistema.

Um sistema armazena um valor que pode ser alterado durante a execução da simulação.

7.11.2 Constructor & Destructor Documentation

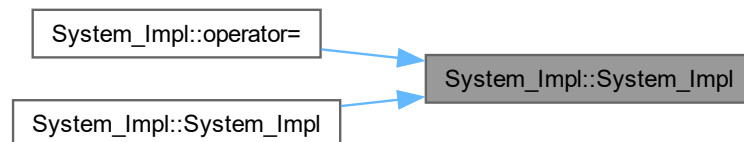
7.11.2.1 System_Impl() [1/3]

System_Impl::System_Impl ()

Construtor padrão da classe [System](#).

Construtor padrão da classe [System_Impl](#).

Inicializa o sistema com valor igual a 0.0 e nome vazio. Here is the caller graph for this function:



7.11.2.2 System_Impl() [2/3]

System_Impl::System_Impl (
 const string & name,
 double value)

Construtor da classe [System](#).

Construtor parametrizado da classe [System_Impl](#).

Parameters

name	Nome do sistema.
value	Valor inicial do sistema.

7.11.2.3 System_Impl() [3/3]

System_Impl::System_Impl (
 const [System_Impl](#) & other)

Construtor de cópia privado.

Construtor de cópia da classe [System_Impl](#).

Impede cópia da classe [System](#).

Parameters

other	Outro objeto System .
other	Objeto System_Impl a ser copiado.

Here is the call graph for this function:



7.11.2.4 ~System_Impl()

System_Impl::~~System_Impl () [virtual]

Destrutor virtual da classe [System](#).

Destrutor da classe [System_Impl](#).

7.11.3 Member Function Documentation

7.11.3.1 getName()

string System_Impl::getName () const [virtual]

Retorna o nome do sistema.

Returns

Nome do sistema.

Implements [System](#).

7.11.3.2 getValue()

double System_Impl::getValue () const [virtual]

Retorna o valor atual do sistema.

Retorna o valor atual armazenado no sistema.

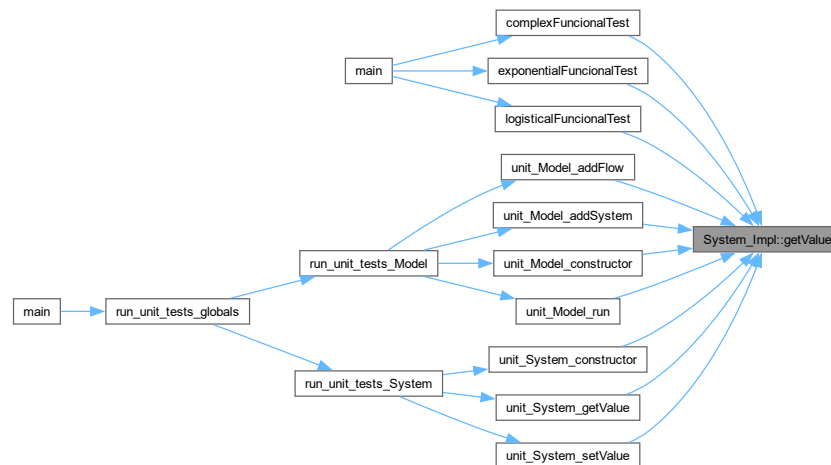
Returns

Valor armazenado no sistema.

Valor do sistema.

Implements [System](#).

Here is the caller graph for this function:



7.11.3.3 operator=()

[System_Impl](#) & [System_Impl::operator=](#) (
const [System_Impl](#) & other)

Operador de atribuição privado.

Operador de atribuição da classe [System_Impl](#).

Impede atribuição entre objetos [System](#).

Parameters

other	Outro objeto System .
-------	---------------------------------------

Returns

Referência para o próprio objeto.

Copia nome e valor de outro objeto [System_Impl](#).

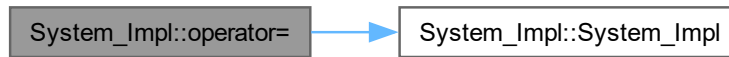
Parameters

other	Objeto System_Impl a ser atribuído.
-------	---

Returns

Referência para o objeto atual.

Here is the call graph for this function:



7.11.3.4 setName()

```
void System_Impl::setName (
    const string & name) [virtual]
```

Define o nome do sistema.

Parameters

name	Novo nome do sistema.
------	-----------------------

Implements [System](#).

7.11.3.5 setValue()

```
void System_Impl::setValue (
    double value) [virtual]
```

Define o valor do sistema.

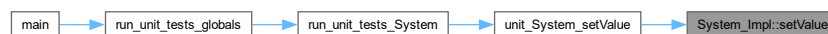
Define o valor armazenado no sistema.

Parameters

value	Novo valor do sistema.
-------	------------------------

Implements [System](#).

Here is the caller graph for this function:



7.11.4 Field Documentation

7.11.4.1 name

```
string System_Impl::name [private]
```

Nome do sistema.

7.11.4.2 value

```
double System_Impl::value [private]
```

Valor armazenado pelo sistema.

The documentation for this class was generated from the following files:

- [include/system_impl.h](#)
- [src/system_impl.cpp](#)

Chapter 8

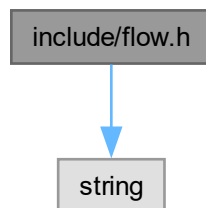
File Documentation

8.1 include/flow.h File Reference

Declaração da interface abstrata [Flow](#).

`#include <string>`

Include dependency graph for flow.h:



Data Structures

- class [Flow](#)

Interface abstrata que representa um fluxo entre dois sistemas.

8.1.1 Detailed Description

Declaração da interface abstrata [Flow](#).

8.2 flow.h

[Go to the documentation of this file.](#)

```
00001 #ifndef FLOW_H
00002 #define FLOW_H
00003
00004 #include <string>
00005
00006 using namespace std;
00007
00008 class System;
00009
00014
00023 class Flow {
00024 public:
00025
00029     virtual ~Flow() = default;
```

```

00030
00036     virtual string getName() const = 0;
00037
00043     virtual void setName(const string& name) = 0;
00044
00050     virtual System* getSource() const = 0;
00051
00057     virtual void setSource(System* source) = 0;
00058
00064     virtual System* getTarget() const = 0;
00065
00071     virtual void setTarget(System* target) = 0;
00072
00082     virtual double execute() = 0;
00083 };
00084
00085 #endif

```

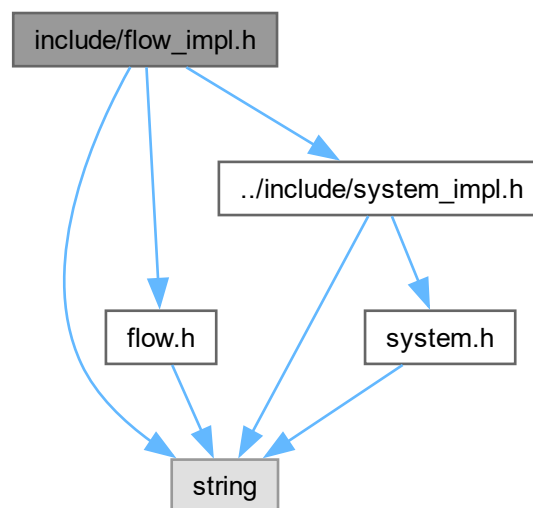
8.3 include/flow_impl.h File Reference

```
#include <string>
```

```
#include "flow.h"
```

```
#include "../include/system_impl.h"
```

Include dependency graph for flow_impl.h:



Data Structures

- class [Flow_Impl](#)

8.4 flow_impl.h

[Go to the documentation of this file.](#)

```

00001 #ifndef FLOW_IMPL_H
00002 #define FLOW_IMPL_H
00003
00004 #include <string>
00005 #include "flow.h"
00006 #include "../include/system_impl.h"
00007 using namespace std;
00008
00009 class System;

```

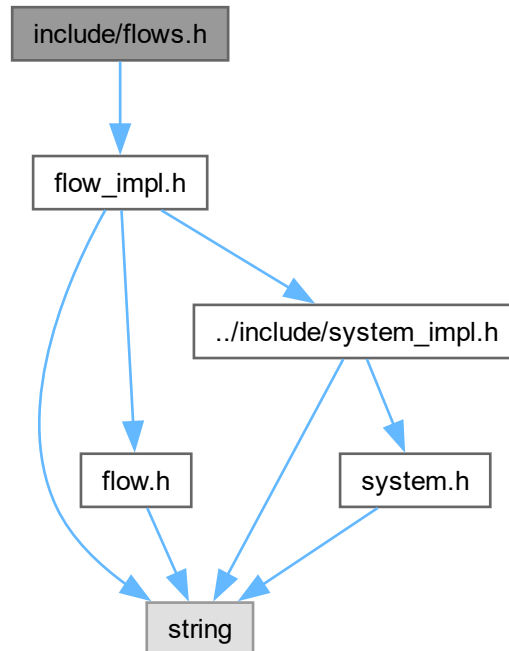
```
00010
00011 class Flow_Impl : public Flow {
00012 protected:
00013     string name;
00014     System* source;
00015     System* target;
00016
00017 public:
00018
00019     // Forma canônica
00020     Flow_Impl();
00021
00022     Flow_Impl(const string& name,
00023               System* source,
00024               System* target);
00025
00026     Flow_Impl(const Flow_Impl& other);
00027
00028     virtual ~Flow_Impl();
00029
00030     Flow_Impl& operator=(const Flow_Impl& other);
00031
00032     // Getters e setters
00033     string getName() const;
00034
00035     void setName(const string& name);
00036
00037     System* getSource() const;
00038
00039     void setSource(System* source);
00040
00041     System* getTarget() const;
00042
00043     void setTarget(System* target);
00044
00045     // Método abstrato
00046     virtual double execute() = 0;
00047
00048 };
00049 #endif
```

8.5 include/flows.h File Reference

Declaração das classes de fluxo concretas: exponencial, logístico e complexo.

```
#include "flow_impl.h"
```

Include dependency graph for flows.h:



Data Structures

- class [FlowExponencial](#)
Fluxo baseado em crescimento exponencial.
- class [FlowLogistico](#)
Fluxo baseado em crescimento logístico.
- class [FlowComplexo](#)
Fluxo com equação de múltiplas interações entre sistemas.

8.5.1 Detailed Description

Declaração das classes de fluxo concretas: exponencial, logístico e complexo.

8.6 flows.h

[Go to the documentation of this file.](#)

```

00001 #ifndef FLOWS_H
00002 #define FLOWS_H
00003
00004 #include "flow_impl.h"
00005
00010
00019 class FlowExponencial : public Flow_Impl {
00020 public:
00021
00025     FlowExponencial();
00026
00034     FlowExponencial(
00035         const string& name,
00036         System* source,

```



```

00037     System* target
00038 );
00039
00045 FlowExponencial(
00046     const FlowExponencial& other
00047 );
00048
00052 virtual ~FlowExponencial();
00053
00060 FlowExponencial& operator=(
00061     const FlowExponencial& other
00062 );
00063
00073 double execute() override;
00074 };
00075
00084 class FlowLogistico : public Flow__Impl {
00085 public:
00086
00090     FlowLogistico();
00091
00099     FlowLogistico(
00100         const string& name,
00101         System* source,
00102         System* target
00103     );
00104
00110     FlowLogistico(
00111         const FlowLogistico& other
00112     );
00113
00117     virtual ~FlowLogistico();
00118
00125     FlowLogistico& operator=(
00126         const FlowLogistico& other
00127     );
00128
00138     double execute() override;
00139 };
00140
00149 class FlowComplexo : public Flow__Impl {
00150 public:
00151
00155     FlowComplexo();
00156
00164     FlowComplexo(
00165         const string& name,
00166         System* source,
00167         System* target
00168     );
00169
00175     FlowComplexo(
00176         const FlowComplexo& other
00177     );
00178
00182     virtual ~FlowComplexo();
00183
00190     FlowComplexo& operator=(
00191         const FlowComplexo& other
00192     );
00193
00204     double execute() override;
00205 };
00206
00207 #endif

```

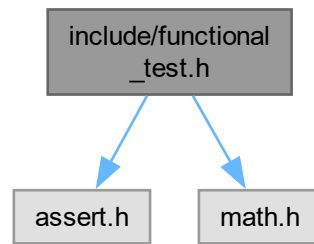
8.7 include/functional_test.h File Reference

Declaração dos testes funcionais dos modelos de simulação.

```
#include <assert.h>
```

```
#include <math.h>
```

Include dependency graph for functional_test.h:



Functions

- void [exponentialFuncionalTest](#) ()
Executa o teste funcional do modelo exponencial.
- void [logisticalFuncionalTest](#) ()
Executa o teste funcional do modelo logístico.
- void [complexFuncionalTest](#) ()
Executa o teste funcional do modelo complexo.

8.7.1 Detailed Description

Declaração dos testes funcionais dos modelos de simulação.

Este arquivo contém os protótipos das funções de teste funcional responsáveis por verificar o comportamento esperado dos modelos exponencial, logístico e complexo após a execução da simulação.

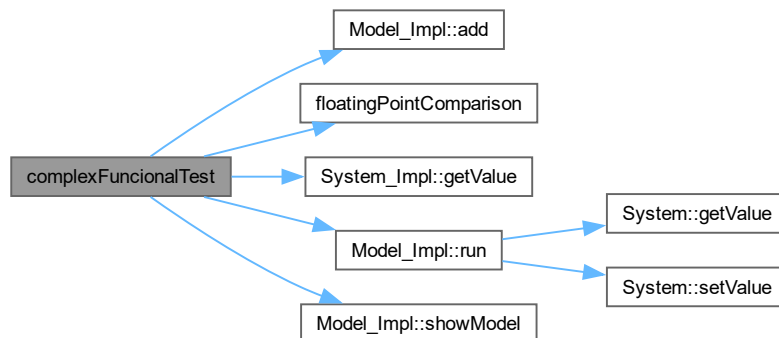
8.7.2 Function Documentation

8.7.2.1 complexFuncionalTest()

void complexFuncionalTest ()

Executa o teste funcional do modelo complexo.

Cria um modelo com múltiplos sistemas e fluxos interligados, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais de cada sistema correspondem aos resultados esperados. Here is the call graph for this function:



Here is the caller graph for this function:

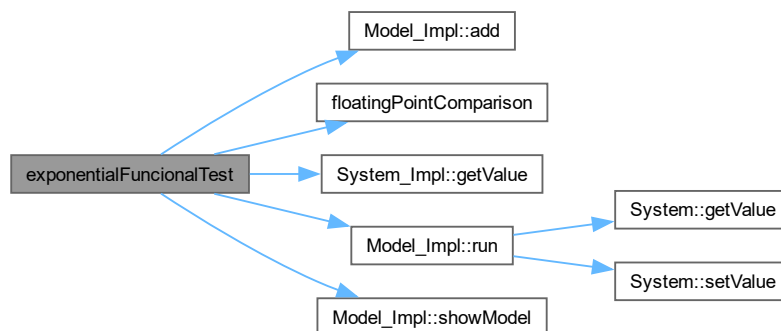


8.7.2.2 exponentialFuncionalTest()

void exponentialFuncionalTest ()

Executa o teste funcional do modelo exponencial.

Cria um modelo com dois sistemas e um fluxo exponencial, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais dos sistemas correspondem aos resultados esperados analiticamente. Here is the call graph for this function:



Here is the caller graph for this function:



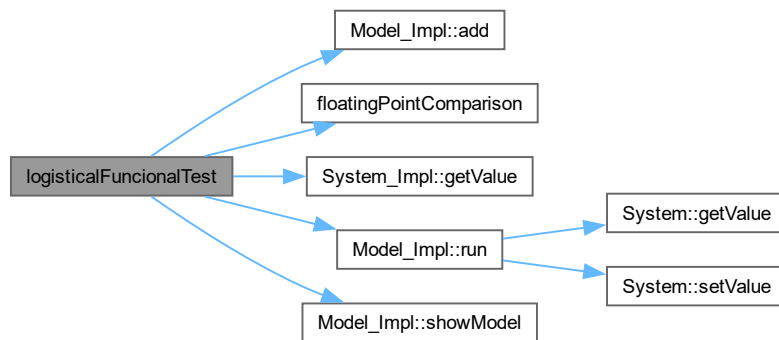
8.7.2.3 logisticalFuncionalTest()

void logisticalFuncionalTest ()

Executa o teste funcional do modelo logístico.

Cria um modelo com dois sistemas e um fluxo logístico, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais dos sistemas correspondem aos resultados esperados

analiticamente. Here is the call graph for this function:



Here is the caller graph for this function:



8.8 functional_test.h

[Go to the documentation of this file.](#)

```

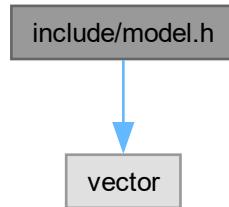
00001 #ifndef FUNCTIONAL_TESTS_H
00002 #define FUNCTIONAL_TESTS_H
00003
00004 #include <assert.h>
00005 #include <math.h>
00006
00015
00024 void exponentialFuncionalTest();
00025
00034 void logisticalFuncionalTest();
00035
00044 void complexFuncionalTest();
00045
00046 #endif
  
```

8.9 include/model.h File Reference

Declaração da interface abstrata [Model](#).

```
#include <vector>
```

Include dependency graph for model.h:



Data Structures

- class [Model](#)

Interface abstrata que representa um modelo de dinâmica de sistemas.

8.9.1 Detailed Description

Declaração da interface abstrata [Model](#).

8.10 model.h

[Go to the documentation of this file.](#)

```

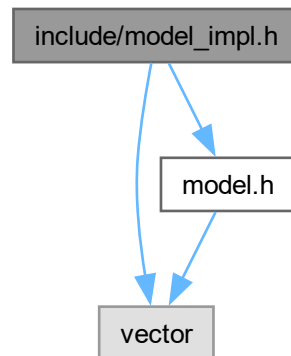
00001 #ifndef MODEL_H
00002 #define MODEL_H
00003
00004 #include <vector>
00005
00006 using namespace std;
00007
00012
00013 // Forward declarations
00014 class System;
00015 class Flow;
00016
00026 class Model {
00027 public:
00028
00032     virtual ~Model() = default;
00033
00039     virtual void add(System* s) = 0;
00040
00046     virtual void add(Flow* f) = 0;
00047
00058     virtual bool run(int t_init, int t_final) = 0;
00059
00063     virtual void showModel() const = 0;
00064 };
00065
00066 #endif
  
```

8.11 include/model_impl.h File Reference

```
#include <vector>
```

```
#include "model.h"
```

Include dependency graph for model_impl.h:



Data Structures

- class `Model_Impl`
Classe responsável por representar um modelo de simulação.

8.12 model_impl.h

[Go to the documentation of this file.](#)

```

00001 #ifndef MODEL_IMPL_H
00002 #define MODEL_IMPL_H
00003
00004 #include <vector>
00005 #include "model.h"
00006 using namespace std;
00007
00008 // Forward declaration
00009 class System;
00010 class Flow;
00011
00019 class Model_Impl : public Model {
00020 private:
00021
00025     vector<System*> systems;
00026
00030     vector<Flow*> flows;
00031
00032 public:
00033
00034     // Forma canônica
00038     Model_Impl();
00039
00043     virtual ~Model_Impl();
00044
00045     // Métodos da UML
00051     void add(System* s);
00052
00058     void add(Flow* f);
00059
00069     bool run(int t_init, int t_final);
00070
00074     void showModel() const;
00075
00076 private:
00077
00085     Model_Impl(const Model_Impl& other);
00086
00095     Model_Impl& operator=(const Model_Impl& other);
00096
00097 };
00098

```

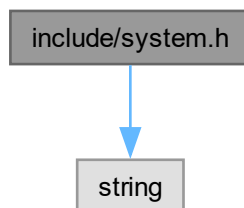
```
00099 #endif
```

8.13 include/system.h File Reference

Declaração da interface abstrata [System](#).

```
#include <string>
```

Include dependency graph for system.h:



Data Structures

- class [System](#)

Interface abstrata que representa um sistema em um modelo de simulação.

8.13.1 Detailed Description

Declaração da interface abstrata [System](#).

8.14 system.h

[Go to the documentation of this file.](#)

```

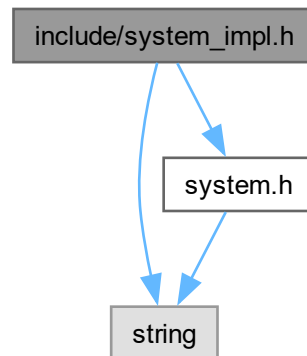
00001 #ifndef SYSTEM_H
00002 #define SYSTEM_H
00003
00004 #include <string>
00005
00006 using namespace std;
00007
00012
00021 class System {
00022 public:
00023
00027     virtual ~System() = default;
00028
00034     virtual string getName() const = 0;
00035
00041     virtual void setName(const string& name) = 0;
00042
00048     virtual double getValue() const = 0;
00049
00055     virtual void setValue(double value) = 0;
00056 };
00057
00058 #endif
  
```

8.15 include/system_impl.h File Reference

```
#include <string>
```

```
#include "system.h"
```

Include dependency graph for system_impl.h:



Data Structures

- class [System_Impl](#)
Classe responsável por representar um sistema.

8.16 system_impl.h

[Go to the documentation of this file.](#)

```

00001 #ifndef SYSTEM_IMPL_H
00002 #define SYSTEM_IMPL_H
00003
00004 #include <string>
00005 #include "system.h"
00006
00007 using namespace std;
00008
00015 class System_Impl : public System {
00016 private:
00017
00021     double value;
00022
00026     string name;
00027
00028 public:
00029     // Forma canônica
00033     System_Impl();
00034
00041     System_Impl(const string& name, double value);
00042
00050     System_Impl(const System_Impl& other);
00051
00055     virtual ~System_Impl();
00056
00065     System_Impl& operator=(const System_Impl& other);
00066
00067     // Métodos da UML
00073     string getName() const;
00074
00080     void setName(const string& name);
00081
00087     double getValue() const;
00088
00094     void setValue(double value);
00095 };
00096
00097 #endif

```


8.17 README.md File Reference

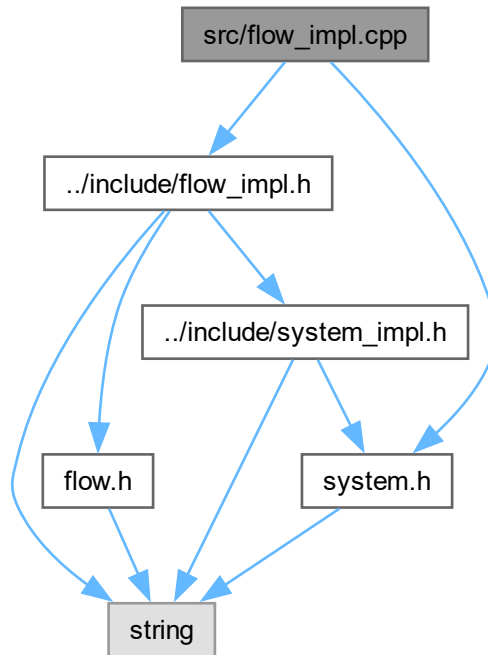
8.18 src/flow_impl.cpp File Reference

Implementação da classe [Flow_Impl](#).

```
#include "../include/flow_impl.h"
```

```
#include "../include/system.h"
```

Include dependency graph for flow_impl.cpp:



8.18.1 Detailed Description

Implementação da classe [Flow_Impl](#).

Este arquivo contém os métodos responsáveis pela manipulação dos fluxos do simulador, incluindo construtores, destrutor, operador de atribuição e getters/setters.

8.19 src/main.cpp File Reference

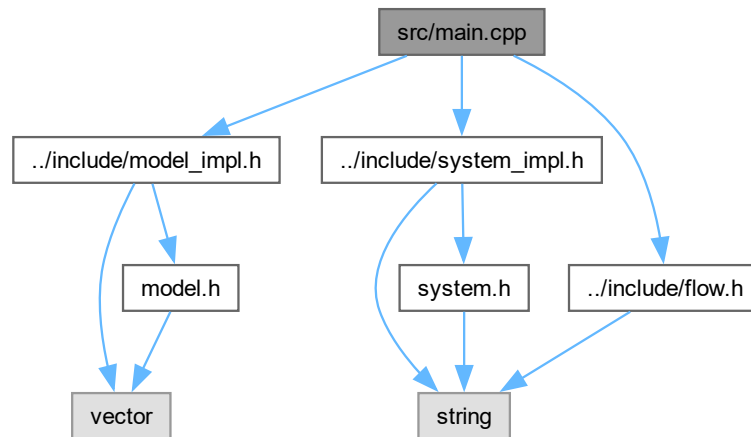
Ponto de entrada principal do simulador.

```
#include "../include/model_impl.h"
```

```
#include "../include/system_impl.h"
```

```
#include "../include/flow.h"
```

Include dependency graph for main.cpp:



Macros

- `#define` [MAIN](#)

Functions

- `int` [main](#) ()
Função principal do simulador.

8.19.1 Detailed Description

Ponto de entrada principal do simulador.

Este arquivo contém a função principal responsável pela inicialização da aplicação do simulador de sistemas dinâmicos.

8.19.2 Macro Definition Documentation

8.19.2.1 MAIN

`#define` MAIN

8.19.3 Function Documentation

8.19.3.1 main()

`int` main ()

Função principal do simulador.

Ponto de entrada da aplicação. Responsável por inicializar e coordenar a execução do simulador.

Returns

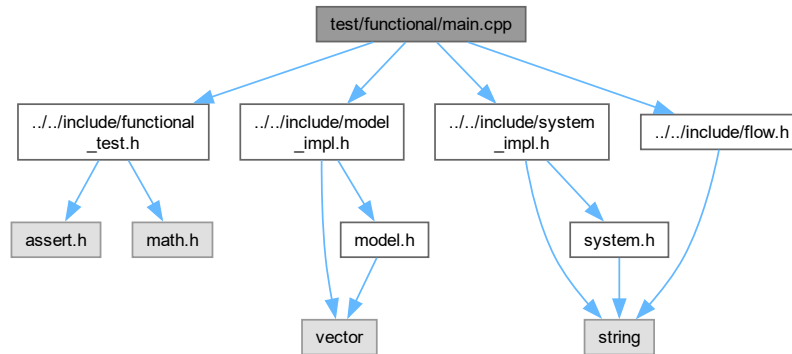
0 caso o programa execute corretamente.

8.20 test/functional/main.cpp File Reference

Ponto de entrada dos testes funcionais do simulador.

```
#include "../include/functional_test.h"
#include "../include/model_impl.h"
#include "../include/system_impl.h"
#include "../include/flow.h"
```

Include dependency graph for main.cpp:



Macros

- `#define` [MAIN_FUNCIONAL_TESTS](#)

Functions

- `int` [main](#) ()
Função principal dos testes funcionais.

8.20.1 Detailed Description

Ponto de entrada dos testes funcionais do simulador.

Este arquivo executa em sequência todos os testes funcionais do simulador de sistemas dinâmicos: exponencial, logístico e complexo. Cada teste verifica se os valores finais da simulação correspondem aos resultados analíticos esperados.

8.20.2 Macro Definition Documentation

8.20.2.1 MAIN_FUNCIONAL_TESTS

```
#define MAIN_FUNCIONAL_TESTS
```

8.20.3 Function Documentation

8.20.3.1 main()

```
int main ()
```

Função principal dos testes funcionais.

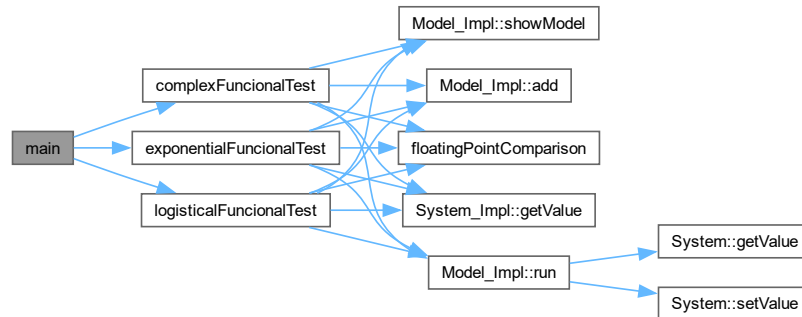
Executa os três testes funcionais do simulador em sequência:

- [exponentialFuncionalTest\(\)](#): verifica o modelo de crescimento exponencial.
- [logisticalFuncionalTest\(\)](#): verifica o modelo de crescimento logístico.
- [complexFuncionalTest\(\)](#): verifica o modelo com múltiplos sistemas e fluxos.

Returns

0 caso todos os testes sejam executados com sucesso.

Here is the call graph for this function:

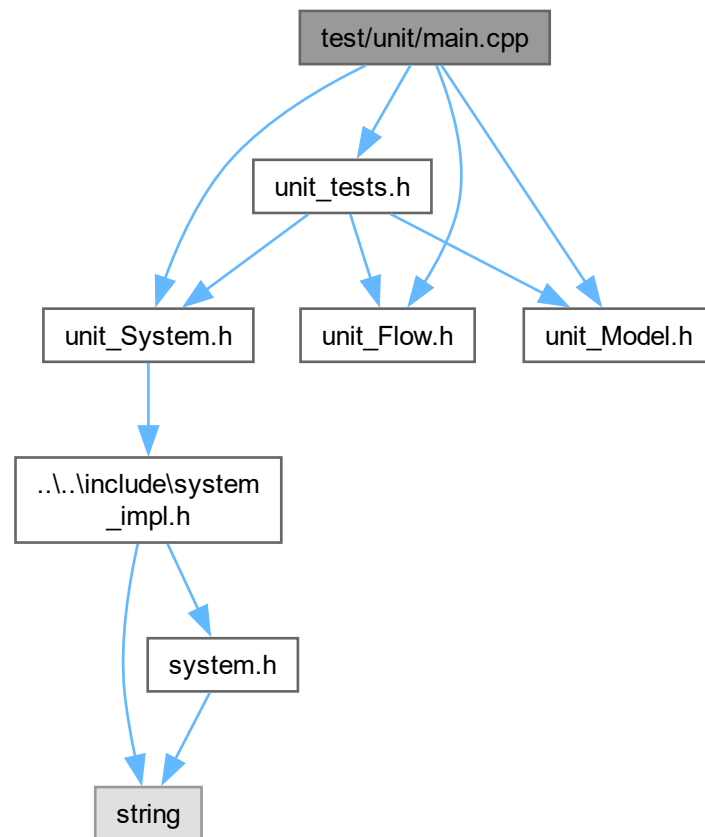


8.21 test/unit/main.cpp File Reference

Ponto de entrada da suíte de testes unitários.

```
#include "unit_tests.h"
#include "unit_System.h"
#include "unit_Flow.h"
#include "unit_Model.h"
```

Include dependency graph for main.cpp:



Functions

- `int main ()`
Função principal do executável de testes unitários.

8.21.1 Detailed Description

Ponto de entrada da suíte de testes unitários.

Arquivo principal responsável por iniciar a execução dos testes unitários do modelo de simulação. Por padrão, executa todos os testes globais via `run_unit_tests_globals()`. As chamadas individuais por módulo (`System`, `Flow`, `Model`) estão disponíveis e podem ser habilitadas removendo os comentários correspondentes.

8.21.2 Function Documentation

8.21.2.1 `main()`

`int main ()`

Função principal do executável de testes unitários.

Invoca a suíte de testes unitários globais. As chamadas individuais por módulo estão comentadas e podem ser ativadas conforme necessário para depuração isolada de cada classe.

Returns

0 em caso de sucesso (todos os asserts aprovados).

Here is the call graph for this function:



8.22 src/model_impl.cpp File Reference

Implementação da classe [Model_Impl](#).

```
#include "../include/model_impl.h"
```

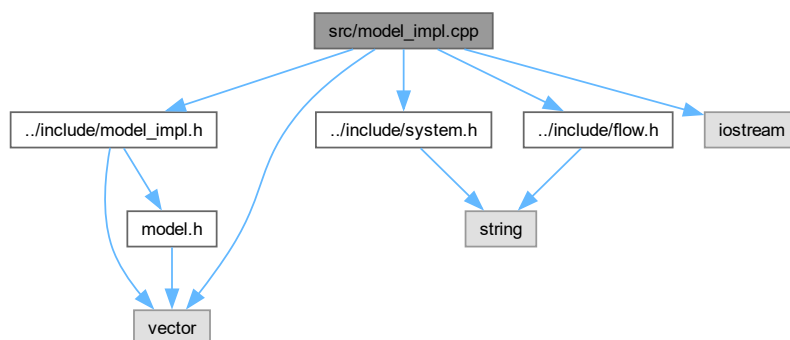
```
#include "../include/system.h"
```

```
#include "../include/flow.h"
```

```
#include <iostream>
```

```
#include <vector>
```

Include dependency graph for model_impl.cpp:



8.22.1 Detailed Description

Implementação da classe [Model_Impl](#).

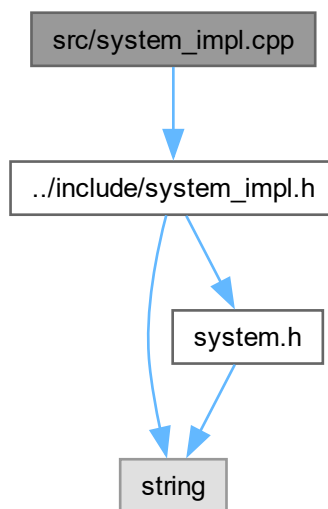
Este arquivo contém os métodos responsáveis pela manipulação e execução dos modelos de simulação de sistemas dinâmicos, incluindo construtores, destrutor, operador de atribuição, adição de sistemas e fluxos, execução da simulação e exibição do estado do modelo.

8.23 src/system_impl.cpp File Reference

Implementação da classe [System_Impl](#).

```
#include "../include/system_impl.h"
```

Include dependency graph for system_impl.cpp:



8.23.1 Detailed Description

Implementação da classe [System_Impl](#).

Este arquivo contém a implementação dos métodos responsáveis pela manipulação dos sistemas do simulador, incluindo construtores, destrutor, operador de atribuição e getters/setters de nome e valor.

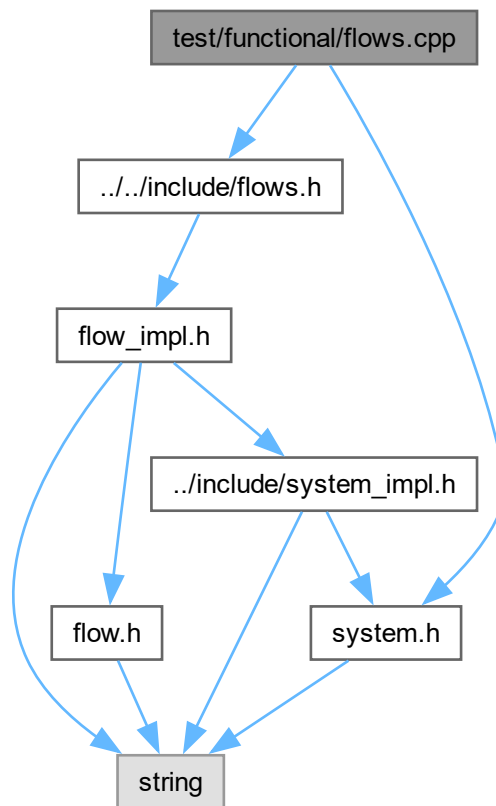
8.24 test/functional/flows.cpp File Reference

Implementação dos fluxos do simulador.

```
#include "../include/flows.h"
```

```
#include "../include/system.h"
```

Include dependency graph for flows.cpp:



8.24.1 Detailed Description

Implementação dos fluxos do simulador.

Este arquivo contém as implementações dos fluxos:

- exponencial
- logístico
- complexo

8.25 test/functional/functional_test.cpp File Reference

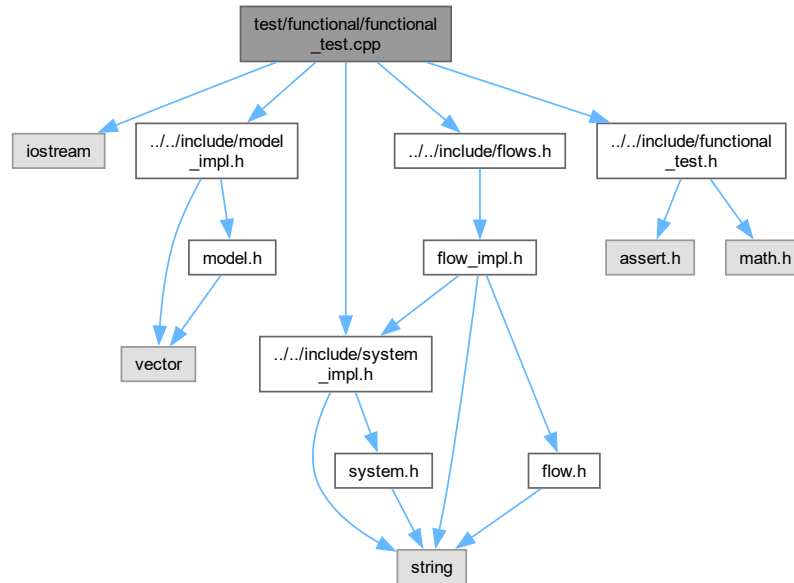
Implementação dos testes funcionais do simulador.

```

#include <iostream>
#include "../..../include/model_impl.h"
#include "../..../include/system_impl.h"
#include "../..../include/flows.h"
#include "../..../include/functional_test.h"

```


Include dependency graph for functional_test.cpp:



Functions

- bool `floatingPointComparison` (double a, double b)
Compara dois números de ponto flutuante com margem de tolerância.
- void `exponentialFuncionalTest` ()
Executa o teste funcional do modelo exponencial.
- void `logisticalFuncionalTest` ()
Executa o teste funcional do modelo logístico.
- void `complexFuncionalTest` ()
Executa o teste funcional do modelo complexo.

8.25.1 Detailed Description

Implementação dos testes funcionais do simulador.

Este arquivo contém os testes funcionais que verificam se os resultados produzidos pela simulação correspondem aos valores analíticos esperados para cada modelo:

- Exponencial
- Logístico
- Complexo

A comparação entre valores de ponto flutuante é feita com uma margem de tolerância de 0.0001 para evitar erros de precisão numérica.

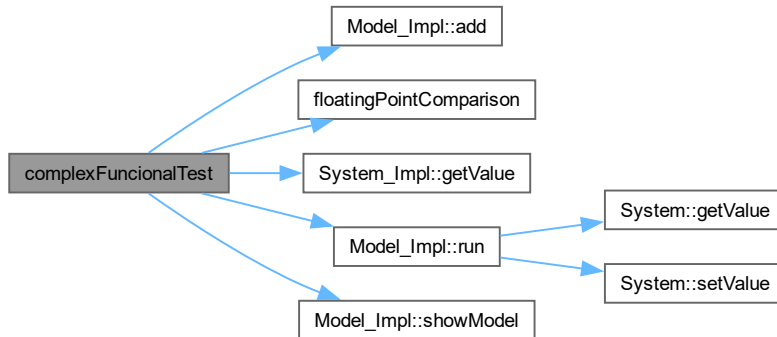
8.25.2 Function Documentation

8.25.2.1 `complexFuncionalTest()`

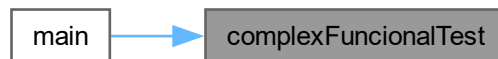
void `complexFuncionalTest` ()

Executa o teste funcional do modelo complexo.

Cria um modelo com múltiplos sistemas e fluxos interligados, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais de cada sistema correspondem aos resultados esperados. Here is the call graph for this function:



Here is the caller graph for this function:

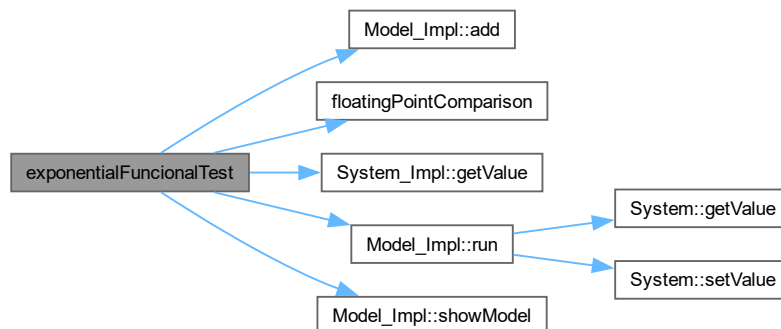


8.25.2.2 exponentialFuncionalTest()

void exponentialFuncionalTest ()

Executa o teste funcional do modelo exponencial.

Cria um modelo com dois sistemas e um fluxo exponencial, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais dos sistemas correspondem aos resultados esperados analiticamente. Here is the call graph for this function:



Here is the caller graph for this function:



8.25.2.3 floatingPointComparison()

```
bool floatingPointComparison (
    double a,
    double b)
```

Compara dois números de ponto flutuante com margem de tolerância.

Utiliza uma margem de erro de 0.0001 para evitar falhas causadas por imprecisões de representação em ponto flutuante.

Parameters

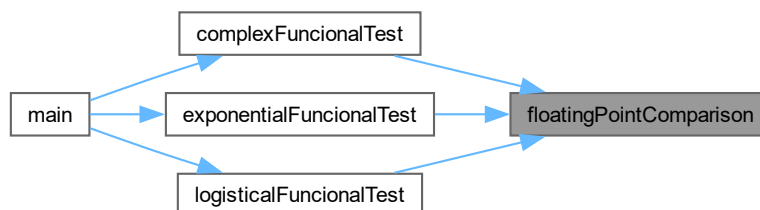
a	Primeiro valor a comparar.
b	Segundo valor a comparar.

Returns

true se a diferença absoluta entre a e b for menor que 0.0001.

false caso contrário.

Here is the caller graph for this function:



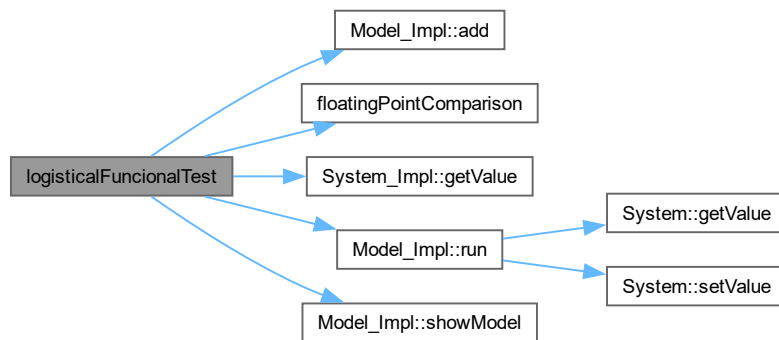
8.25.2.4 logisticalFuncionalTest()

```
void logisticalFuncionalTest ()
```

Executa o teste funcional do modelo logístico.

Cria um modelo com dois sistemas e um fluxo logístico, executa a simulação por um intervalo de tempo pré-definido e verifica, via assert, se os valores finais dos sistemas correspondem aos resultados esperados

analiticamente. Here is the call graph for this function:



Here is the caller graph for this function:



8.26 test/unit/unit_flow.cpp File Reference

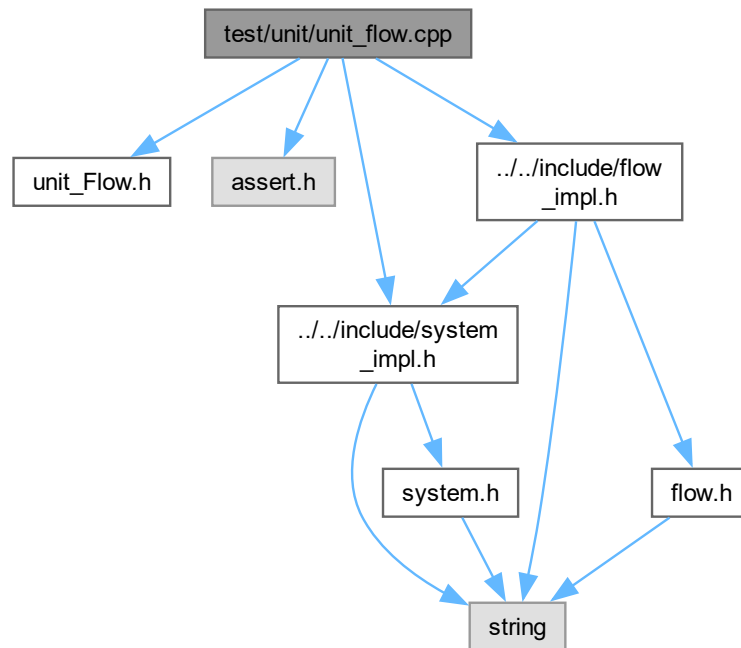
Implementação dos testes unitários da classe [Flow](#).

```

#include "unit_Flow.h"
#include <assert.h>
#include "../include/system_impl.h"
#include "../include/flow_impl.h"

```

Include dependency graph for unit_flow.cpp:



Data Structures

- class `Flow_Test`
Implementação concreta de `Flow_Impl` utilizada nos testes unitários.

Functions

- void `unit_Flow_constructor` (void)
Testa o construtor parametrizado da classe `Flow`.
- void `unit_Flow_destructor` (void)
Testa o destrutor da classe `Flow`.
- void `unit_Flow_getName` (void)
Testa o método `getName()` da classe `Flow`.
- void `unit_Flow_setName` (void)
Testa o método `setName()` da classe `Flow`.
- void `unit_Flow_getSource` (void)
Testa o método `getSource()` da classe `Flow`.
- void `unit_Flow_setSource` (void)
Testa o método `setSource()` da classe `Flow`.
- void `unit_Flow_getTarget` (void)
Testa o método `getTarget()` da classe `Flow`.
- void `unit_Flow_setTarget` (void)
Testa o método `setTarget()` da classe `Flow`.
- void `unit_Flow_execute` (void)
Testa o método `execute()` da classe `Flow`.
- void `run_unit_tests_Flow` (void)
Executa todos os testes unitários da classe `Flow` em sequência.

8.26.1 Detailed Description

Implementação dos testes unitários da classe [Flow](#).

Contém uma classe de fluxo concreta auxiliar ([Flow_Test](#)) utilizada exclusivamente nos testes, além das implementações de cada função de teste unitário declarada em [unit_Flow.h](#).

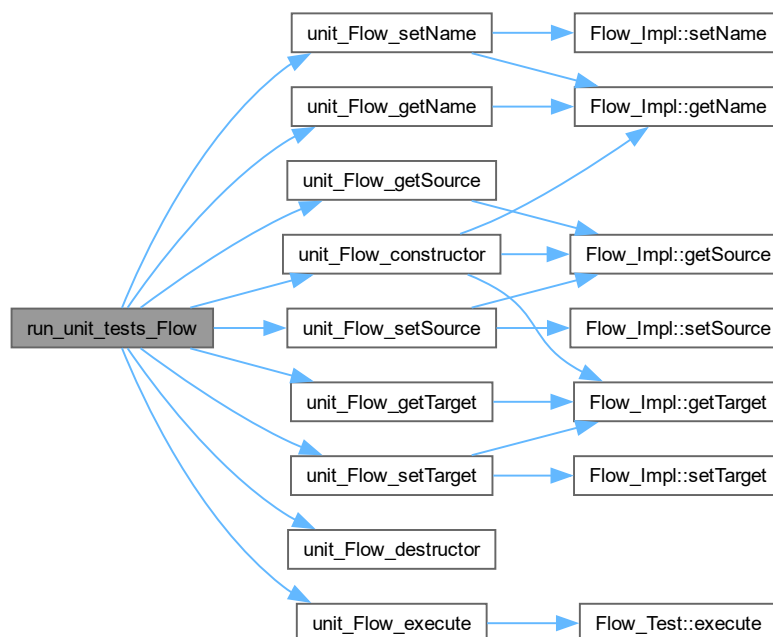
8.26.2 Function Documentation

8.26.2.1 run_unit_tests_Flow()

```
void run_unit_tests_Flow (
    void )
```

Executa todos os testes unitários da classe [Flow](#) em sequência.

Chama cada função de teste unitário da classe [Flow](#) e exibe uma mensagem de confirmação caso todos sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:

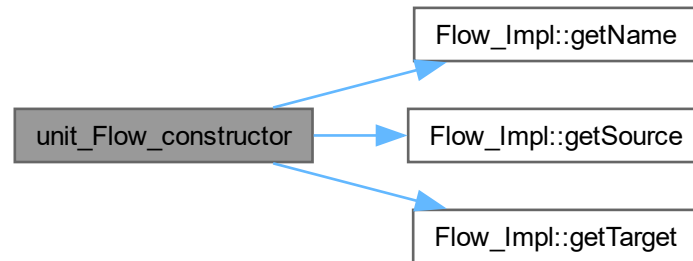


8.26.2.2 unit_Flow_constructor()

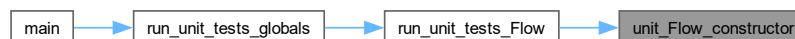
```
void unit_Flow_constructor (
    void )
```

Testa o construtor parametrizado da classe [Flow](#).

Verifica se, ao criar um fluxo com nome, sistema de origem e sistema de destino, os atributos são corretamente inicializados. Here is the call graph for this function:



Here is the caller graph for this function:



8.26.2.3 unit_Flow_destructor()

```
void unit_Flow_destructor (
    void )
```

Testa o destrutor da classe [Flow](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



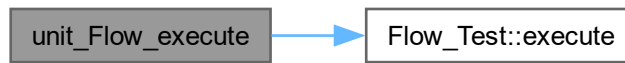
8.26.2.4 unit_Flow_execute()

```
void unit_Flow_execute (
    void )
```

Testa o método `execute()` da classe [Flow](#).

Verifica se o valor retornado por `execute()` corresponde ao resultado esperado para a equação do fluxo de

teste. Here is the call graph for this function:



Here is the caller graph for this function:



8.26.2.5 unit_Flow_getName()

```
void unit_Flow_getName (  
    void )
```

Testa o método `getName()` da classe [Flow](#).

Verifica se o nome retornado corresponde ao valor definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



8.26.2.6 unit_Flow_getSource()

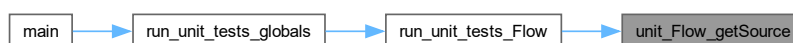
```
void unit_Flow_getSource (  
    void )
```

Testa o método `getSource()` da classe [Flow](#).

Verifica se o ponteiro para o sistema de origem retornado corresponde ao sistema definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



8.26.2.7 unit_Flow_getTarget()

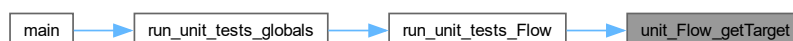
```
void unit_Flow_getTarget (  
    void )
```

Testa o método `getTarget()` da classe [Flow](#).

Verifica se o ponteiro para o sistema de destino retornado corresponde ao sistema definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:

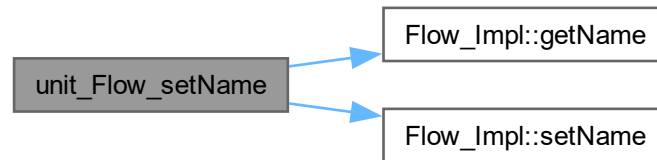


8.26.2.8 unit_Flow_setName()

```
void unit_Flow_setName (  
    void )
```

Testa o método `setName()` da classe [Flow](#).

Verifica se o nome do fluxo é atualizado corretamente após a chamada de setName(). Here is the call graph for this function:



Here is the caller graph for this function:

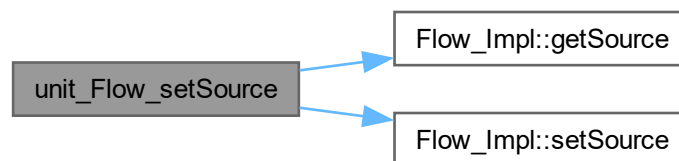


8.26.2.9 unit_Flow_setSource()

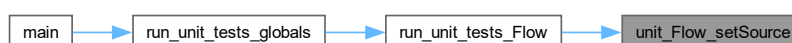
```
void unit_Flow_setSource (
    void )
```

Testa o método `setSource()` da classe [Flow](#).

Verifica se o sistema de origem é atualizado corretamente após a chamada de `setSource()`. Here is the call graph for this function:



Here is the caller graph for this function:

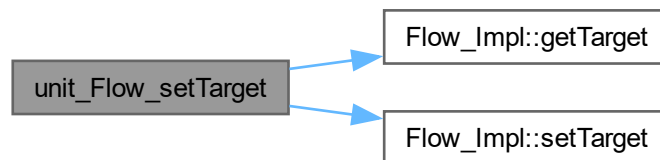


8.26.2.10 unit_Flow_setTarget()

```
void unit_Flow_setTarget (
    void )
```

Testa o método setTarget() da classe [Flow](#).

Verifica se o sistema de destino é atualizado corretamente após a chamada de setTarget(). Here is the call graph for this function:



Here is the caller graph for this function:



8.27 test/unit/unit_flow.h File Reference

Declaração dos testes unitários da classe [Flow](#).

Functions

- void [unit_Flow_constructor](#) (void)
Testa o construtor parametrizado da classe [Flow](#).
- void [unit_Flow_destructor](#) (void)
Testa o destrutor da classe [Flow](#).
- void [unit_Flow_getName](#) (void)
Testa o método getName() da classe [Flow](#).
- void [unit_Flow_setName](#) (void)
Testa o método setName() da classe [Flow](#).
- void [unit_Flow_getSource](#) (void)
Testa o método getSource() da classe [Flow](#).
- void [unit_Flow_setSource](#) (void)
Testa o método setSource() da classe [Flow](#).
- void [unit_Flow_getTarget](#) (void)
Testa o método getTarget() da classe [Flow](#).
- void [unit_Flow_setTarget](#) (void)
Testa o método setTarget() da classe [Flow](#).
- void [unit_Flow_execute](#) (void)
Testa o método execute() da classe [Flow](#).
- void [run_unit_tests_Flow](#) (void)
Executa todos os testes unitários da classe [Flow](#) em sequência.

8.27.1 Detailed Description

Declaração dos testes unitários da classe [Flow](#).

Este arquivo contém os protótipos das funções de teste unitário responsáveis por verificar individualmente cada método da classe [Flow](#) e sua implementação concreta [Flow_Impl](#).

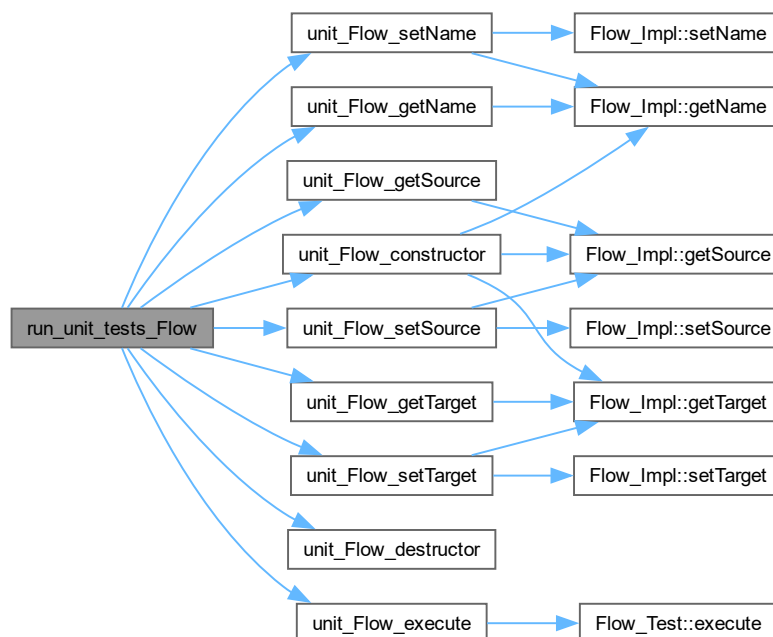
8.27.2 Function Documentation

8.27.2.1 `run_unit_tests_Flow()`

```
void run_unit_tests_Flow (
    void )
```

Executa todos os testes unitários da classe [Flow](#) em sequência.

Chama cada função de teste unitário da classe [Flow](#) e exibe uma mensagem de confirmação caso todos sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:

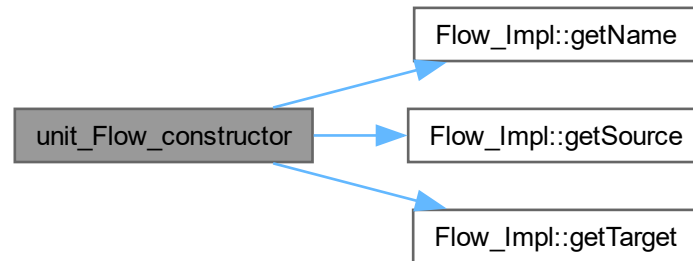


8.27.2.2 `unit_Flow_constructor()`

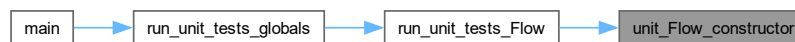
```
void unit_Flow_constructor (
    void )
```

Testa o construtor parametrizado da classe [Flow](#).

Verifica se, ao criar um fluxo com nome, sistema de origem e sistema de destino, os atributos são corretamente inicializados. Here is the call graph for this function:



Here is the caller graph for this function:



8.27.2.3 unit_Flow_destructor()

```
void unit_Flow_destructor (
    void )
```

Testa o destrutor da classe [Flow](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



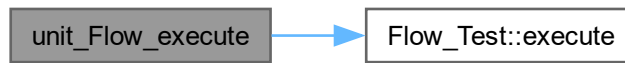
8.27.2.4 unit_Flow_execute()

```
void unit_Flow_execute (
    void )
```

Testa o método `execute()` da classe [Flow](#).

Verifica se o valor retornado por `execute()` corresponde ao resultado esperado para a equação do fluxo de

teste. Here is the call graph for this function:



Here is the caller graph for this function:



8.27.2.5 unit_Flow_getName()

```
void unit_Flow_getName (  
    void )
```

Testa o método `getName()` da classe [Flow](#).

Verifica se o nome retornado corresponde ao valor definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



8.27.2.6 unit_Flow_getSource()

```
void unit_Flow_getSource (  
    void )
```

Testa o método `getSource()` da classe [Flow](#).

Verifica se o ponteiro para o sistema de origem retornado corresponde ao sistema definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



8.27.2.7 unit_Flow_getTarget()

```
void unit_Flow_getTarget (  
    void )
```

Testa o método `getTarget()` da classe [Flow](#).

Verifica se o ponteiro para o sistema de destino retornado corresponde ao sistema definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:

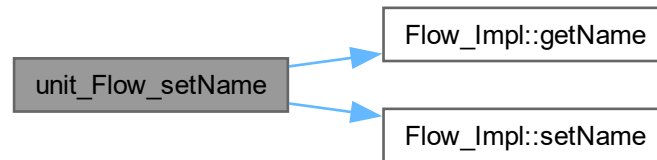


8.27.2.8 unit_Flow_setName()

```
void unit_Flow_setName (  
    void )
```

Testa o método `setName()` da classe [Flow](#).

Verifica se o nome do fluxo é atualizado corretamente após a chamada de setName(). Here is the call graph for this function:



Here is the caller graph for this function:

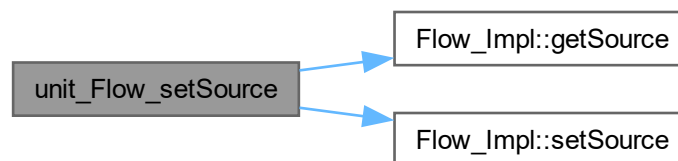


8.27.2.9 unit_Flow_setSource()

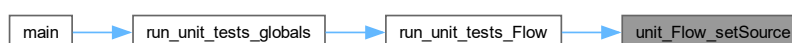
```
void unit_Flow_setSource (  
    void )
```

Testa o método `setSource()` da classe [Flow](#).

Verifica se o sistema de origem é atualizado corretamente após a chamada de `setSource()`. Here is the call graph for this function:



Here is the caller graph for this function:

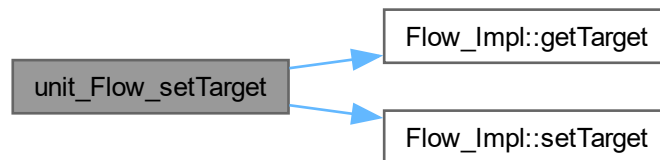


8.27.2.10 unit_Flow_setTarget()

```
void unit_Flow_setTarget (
    void )
```

Testa o método setTarget() da classe [Flow](#).

Verifica se o sistema de destino é atualizado corretamente após a chamada de setTarget(). Here is the call graph for this function:



Here is the caller graph for this function:



8.28 unit_flow.h

[Go to the documentation of this file.](#)

```

00001 #ifndef UNIT_FLOW_H
00002 #define UNIT_FLOW_H
00003
00012
00019 void unit_Flow_constructor(void);
00020
00026 void unit_Flow_destructor(void);
00027
00034 void unit_Flow_getName(void);
00035
00042 void unit_Flow_setName(void);
00043
00050 void unit_Flow_getSource(void);
00051
00058 void unit_Flow_setSource(void);
00059
00066 void unit_Flow_getTarget(void);
00067
00074 void unit_Flow_setTarget(void);
00075
00082 void unit_Flow_execute(void);
00083
00090 void run_unit_tests_Flow(void);
00091
00092 #endif
```

8.29 test/unit/unit_model.cpp File Reference

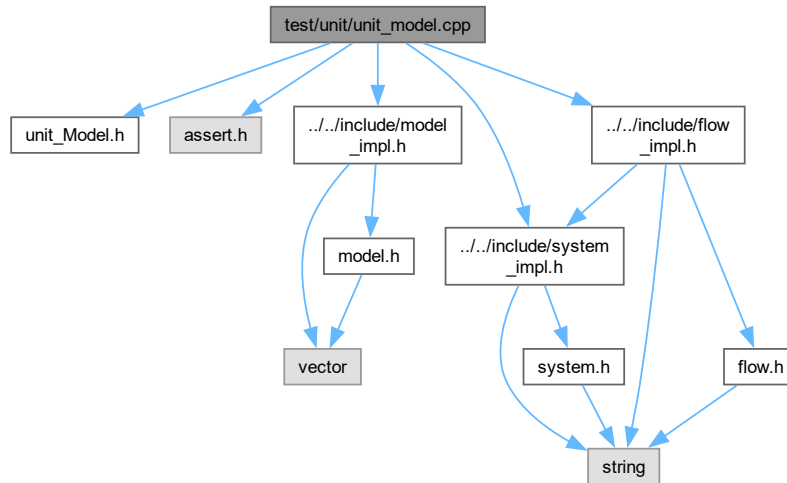
Implementação dos testes unitários da classe [Model](#).

```
#include "unit_Model.h"
```

```
#include <assert.h>
```

```
#include "../include/model_impl.h"
```

```
#include "../include/system_impl.h"
#include "../include/flow_impl.h"
Include dependency graph for unit_model.cpp:
```



Data Structures

- class [FlowTest](#)
Implementação concreta de [Flow_Impl](#) utilizada nos testes do [Model](#).

Functions

- void [unit_Model_constructor](#) (void)
Testa o construtor da classe [Model](#).
- void [unit_Model_destructor](#) (void)
Testa o destrutor da classe [Model](#).
- void [unit_Model_addSystem](#) (void)
Testa o método `add(System*)` da classe [Model](#).
- void [unit_Model_addFlow](#) (void)
Testa o método `add(Flow*)` da classe [Model](#).
- void [unit_Model_run](#) (void)
Testa o método `run()` da classe [Model](#).
- void [unit_Model_showModel](#) (void)
Testa o método `showModel()` da classe [Model](#).
- void [run_unit_tests_Model](#) (void)
Executa todos os testes unitários da classe [Model](#) em sequência.

8.29.1 Detailed Description

Implementação dos testes unitários da classe [Model](#).

Contém uma classe de fluxo concreta auxiliar ([FlowTest](#)) utilizada exclusivamente nos testes, além das implementações de cada função de teste unitário declarada em [unit_Model.h](#).

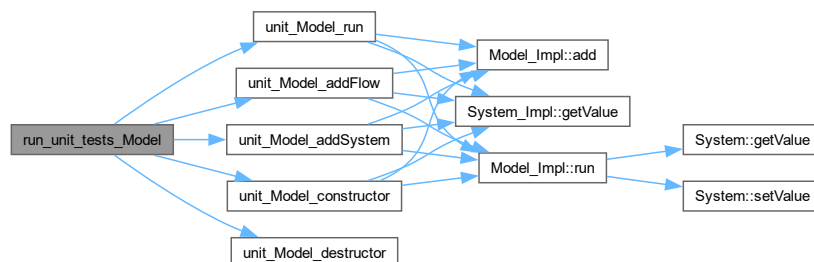
8.29.2 Function Documentation

8.29.2.1 run_unit_tests_Model()

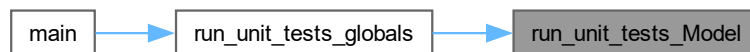
```
void run_unit_tests_Model (
    void )
```

Executa todos os testes unitários da classe [Model](#) em sequência.

Chama cada função de teste unitário da classe [Model](#) e exibe uma mensagem de confirmação caso todos sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:

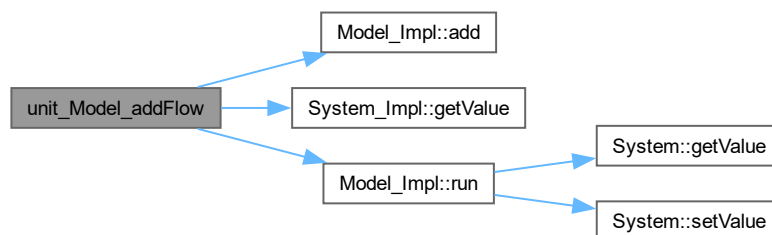


8.29.2.2 unit_Model_addFlow()

```
void unit_Model_addFlow (
    void )
```

Testa o método `add(Flow*)` da classe [Model](#).

Verifica o comportamento do modelo com e sem fluxo adicionado: sem fluxo os sistemas não se alteram; com fluxo a transferência ocorre corretamente a cada passo de tempo. Here is the call graph for this function:



Here is the caller graph for this function:

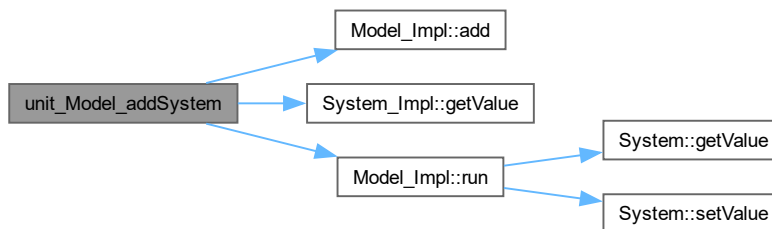


8.29.2.3 unit_Model_addSystem()

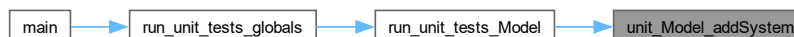
```
void unit_Model_addSystem (
    void )
```

Testa o método `add(System*)` da classe [Model](#).

Verifica se sistemas adicionados ao modelo participam corretamente da simulação, tendo seus valores atualizados após a execução dos fluxos. Here is the call graph for this function:



Here is the caller graph for this function:



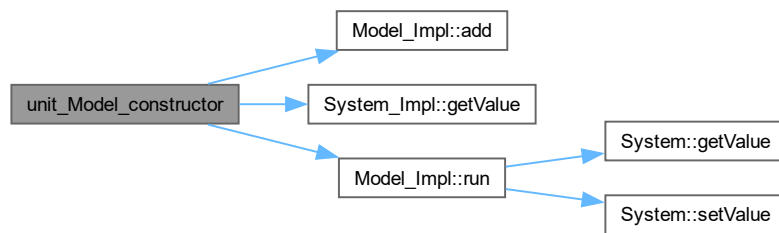
8.29.2.4 unit_Model_constructor()

```
void unit_Model_constructor (
    void )
```

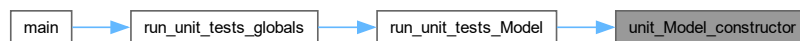
Testa o construtor da classe [Model](#).

Cria um modelo, adiciona sistemas e um fluxo de teste, executa a simulação por um passo e verifica se

os valores dos sistemas foram atualizados corretamente. Here is the call graph for this function:



Here is the caller graph for this function:

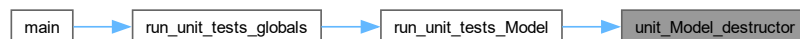


8.29.2.5 unit_Model_destructor()

```
void unit_Model_destructor (
    void )
```

Testa o destrutor da classe [Model](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



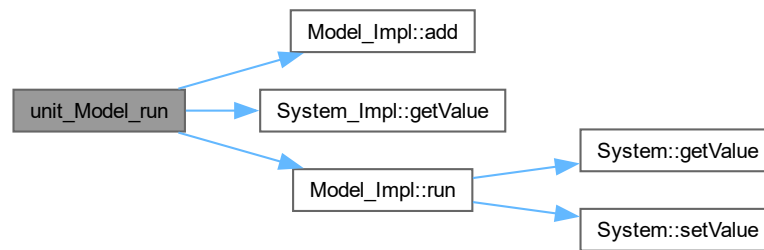
8.29.2.6 unit_Model_run()

```
void unit_Model_run (
    void )
```

Testa o método `run()` da classe [Model](#).

Executa a simulação por múltiplos passos de tempo e verifica se os valores finais dos sistemas correspondem

ao total acumulado de transferências esperado. Here is the call graph for this function:



Here is the caller graph for this function:

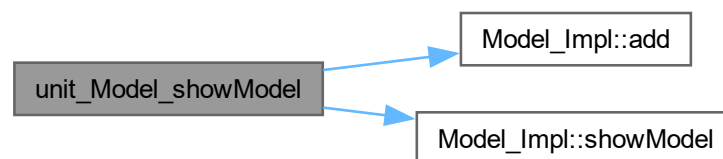


8.29.2.7 unit_Model_showModel()

```
void unit_Model_showModel (
    void )
```

Testa o método `showModel()` da classe [Model](#).

Verifica se a exibição das informações do modelo é executada sem erros. A correção visual da saída deve ser verificada manualmente. Here is the call graph for this function:



8.30 test/unit/unit_model.h File Reference

Declaração dos testes unitários da classe [Model](#).

Functions

- void [unit_Model_constructor](#) (void)
Testa o construtor da classe [Model](#).
- void [unit_Model_destructor](#) (void)

- Testa o destrutor da classe [Model](#).
- void [unit_Model_addSystem](#) (void)
Testa o método add(System*) da classe [Model](#).
- void [unit_Model_addFlow](#) (void)
Testa o método add(Flow*) da classe [Model](#).
- void [unit_Model_run](#) (void)
Testa o método run() da classe [Model](#).
- void [unit_Model_showModel](#) (void)
Testa o método showModel() da classe [Model](#).
- void [run_unit_tests_Model](#) (void)
Executa todos os testes unitários da classe [Model](#) em sequência.

8.30.1 Detailed Description

Declaração dos testes unitários da classe [Model](#).

Este arquivo contém os protótipos das funções de teste unitário responsáveis por verificar individualmente cada método da classe [Model](#) e sua implementação concreta [Model_Impl](#).

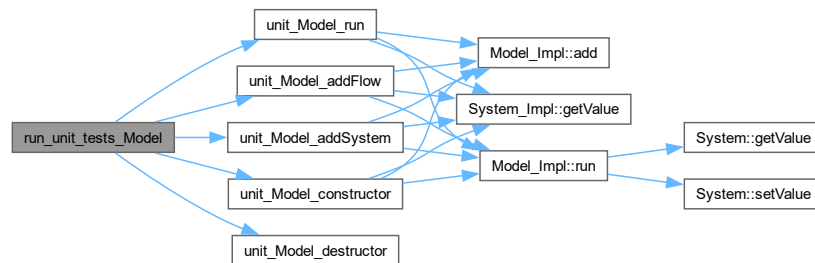
8.30.2 Function Documentation

8.30.2.1 run_unit_tests_Model()

```
void run_unit_tests_Model (
    void )
```

Executa todos os testes unitários da classe [Model](#) em sequência.

Chama cada função de teste unitário da classe [Model](#) e exibe uma mensagem de confirmação caso todos sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:

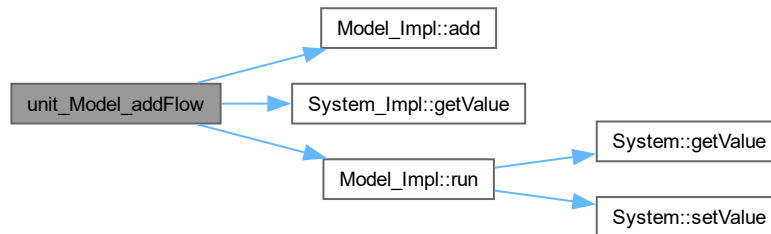


8.30.2.2 unit_Model_addFlow()

```
void unit_Model_addFlow (
    void )
```

Testa o método `add(Flow*)` da classe [Model](#).

Verifica o comportamento do modelo com e sem fluxo adicionado: sem fluxo os sistemas não se alteram; com fluxo a transferência ocorre corretamente a cada passo de tempo. Here is the call graph for this function:



Here is the caller graph for this function:

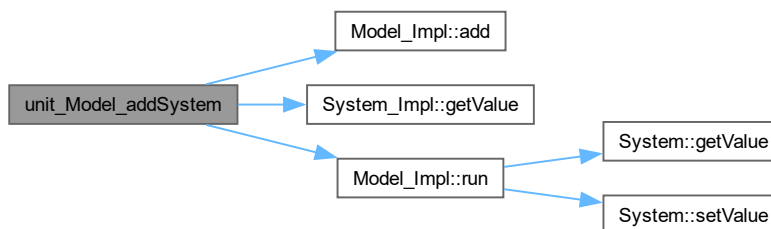


8.30.2.3 unit_Model_addSystem()

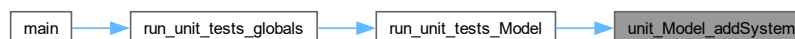
```
void unit_Model_addSystem (
    void )
```

Testa o método `add(System*)` da classe [Model](#).

Verifica se sistemas adicionados ao modelo participam corretamente da simulação, tendo seus valores atualizados após a execução dos fluxos. Here is the call graph for this function:



Here is the caller graph for this function:

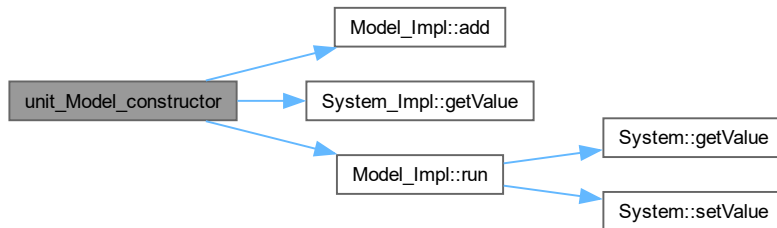


8.30.2.4 unit_Model_constructor()

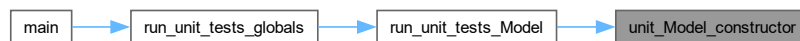
```
void unit_Model_constructor (
    void )
```

Testa o construtor da classe [Model](#).

Cria um modelo, adiciona sistemas e um fluxo de teste, executa a simulação por um passo e verifica se os valores dos sistemas foram atualizados corretamente. Here is the call graph for this function:



Here is the caller graph for this function:

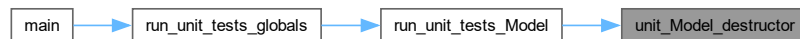


8.30.2.5 unit_Model_destructor()

```
void unit_Model_destructor (
    void )
```

Testa o destrutor da classe [Model](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



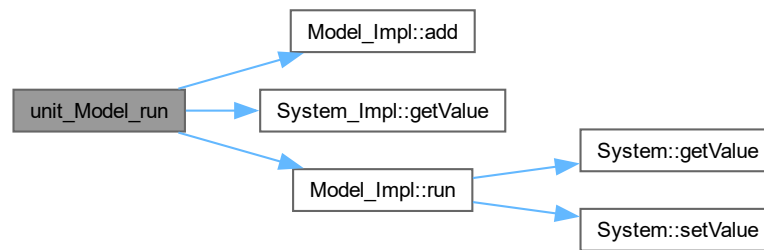
8.30.2.6 unit_Model_run()

```
void unit_Model_run (
    void )
```

Testa o método `run()` da classe [Model](#).

Executa a simulação por múltiplos passos de tempo e verifica se os valores finais dos sistemas correspondem

ao total acumulado de transferências esperado. Here is the call graph for this function:



Here is the caller graph for this function:

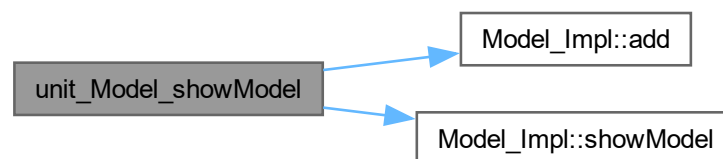


8.30.2.7 unit_Model_showModel()

```
void unit_Model_showModel (
    void )
```

Testa o método `showModel()` da classe [Model](#).

Verifica se a exibição das informações do modelo é executada sem erros. A correção visual da saída deve ser verificada manualmente. Here is the call graph for this function:



8.31 unit_model.h

[Go to the documentation of this file.](#)

```
00001 #ifndef UNIT_MODEL_H
00002 #define UNIT_MODEL_H
00003
00012
00020 void unit_Model_constructor(void);
00021
00027 void unit_Model_destructor(void);
00028
00035 void unit_Model_addSystem(void);
```

```

00036
00044 void unit_Model_addFlow(void);
00045
00053 void unit_Model_run(void);
00054
00061 void unit_Model_showModel(void);
00062
00069 void run_unit_tests_Model(void);
00070
00071 #endif

```

8.32 test/unit/unit_system.cpp File Reference

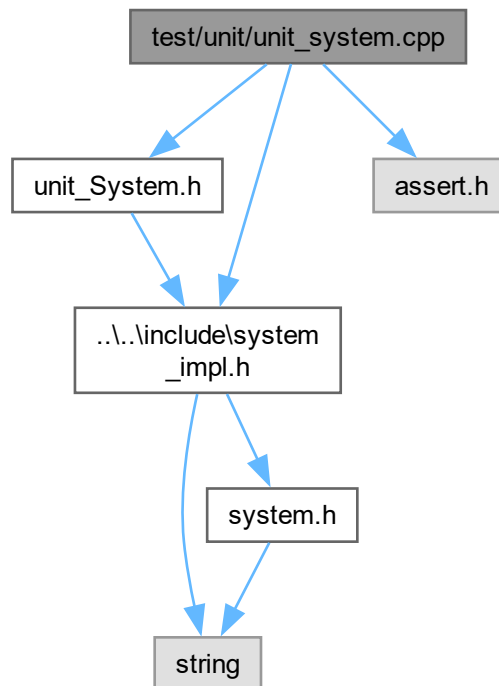
Implementação dos testes unitários da classe [System](#).

#include "unit_System.h"

#include <assert.h>

#include "../include/system_impl.h"

Include dependency graph for unit_system.cpp:



Functions

- void `unit_System_constructor` (void)
Testa o construtor parametrizado da classe [System](#).
- void `unit_System_destructor` (void)
Testa o destrutor da classe [System](#).
- void `unit_System_getValue` (void)
Testa o método `getValue()` da classe [System](#).
- void `unit_System_setValue` (void)
Testa o método `setValue()` da classe [System](#).
- void `run_unit_tests_System` (void)

Executa todos os testes unitários da classe [System](#) em sequência.

8.32.1 Detailed Description

Implementação dos testes unitários da classe [System](#).

Contém as implementações de cada função de teste unitário declarada em [unit_System.h](#), verificando o comportamento da classe [System_Impl](#) para construção, destruição e manipulação de valores.

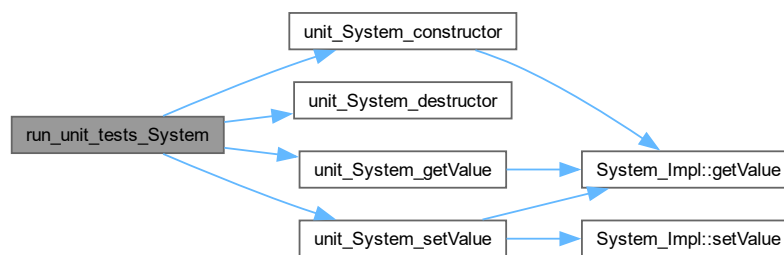
8.32.2 Function Documentation

8.32.2.1 run_unit_tests_System()

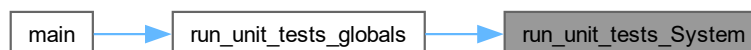
```
void run_unit_tests_System (
    void )
```

Executa todos os testes unitários da classe [System](#) em sequência.

Chama cada função de teste unitário da classe [System](#) e exibe uma mensagem de confirmação caso todos sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:



8.32.2.2 unit_System_constructor()

```
void unit_System_constructor (
    void )
```

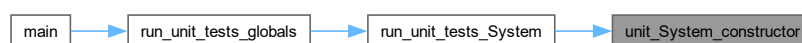
Testa o construtor parametrizado da classe [System](#).

Verifica se, ao criar sistemas com valores iniciais distintos, o valor armazenado corresponde ao informado

na construção. Here is the call graph for this function:



Here is the caller graph for this function:

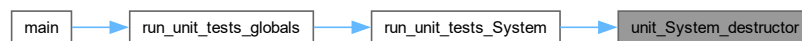


8.32.2.3 unit_System_destructor()

```
void unit_System_destructor (  
    void )
```

Testa o destrutor da classe [System](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



8.32.2.4 unit_System_getValue()

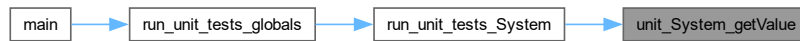
```
void unit_System_getValue (  
    void )
```

Testa o método `getValue()` da classe [System](#).

Verifica se o valor retornado corresponde ao valor definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:

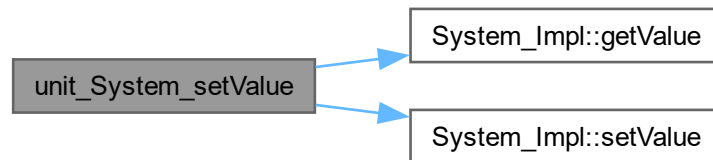


8.32.2.5 unit_System_setValue()

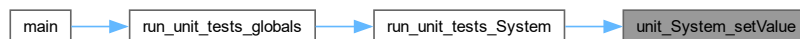
```
void unit_System_setValue (
    void )
```

Testa o método `setValue()` da classe [System](#).

Verifica se o valor do sistema é atualizado corretamente após a chamada de `setValue()`, sendo refletido por uma chamada subsequente a `getValue()`. Here is the call graph for this function:



Here is the caller graph for this function:

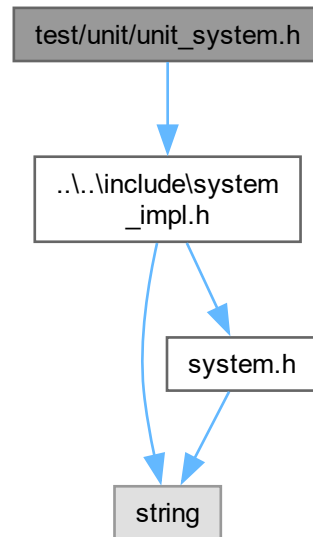


8.33 test/unit/unit_system.h File Reference

Declaração dos testes unitários da classe [System](#).

```
#include "..\..\include\system_impl.h"
```

Include dependency graph for unit_system.h:



Functions

- void `unit_System_constructor` (void)
Testa o construtor parametrizado da classe `System`.
- void `unit_System_destructor` (void)
Testa o destrutor da classe `System`.
- void `unit_System_getValue` (void)
Testa o método `getValue()` da classe `System`.
- void `unit_System_setValue` (void)
Testa o método `setValue()` da classe `System`.
- void `run_unit_tests_System` (void)
Executa todos os testes unitários da classe `System` em sequência.

8.33.1 Detailed Description

Declaração dos testes unitários da classe `System`.

Este arquivo contém os protótipos das funções de teste unitário responsáveis por verificar individualmente cada método da classe `System` e sua implementação concreta `System_Impl`.

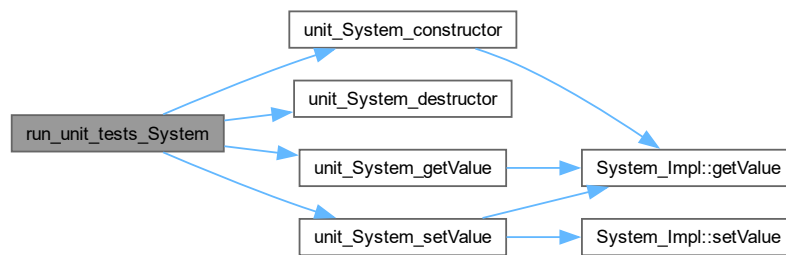
8.33.2 Function Documentation

8.33.2.1 `run_unit_tests_System()`

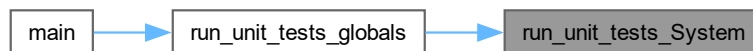
```
void run_unit_tests_System (
    void )
```

Executa todos os testes unitários da classe `System` em sequência.

Chama cada função de teste unitário da classe [System](#) e exibe uma mensagem de confirmação caso todos sejam aprovados. Here is the call graph for this function:



Here is the caller graph for this function:



8.33.2.2 unit_System_constructor()

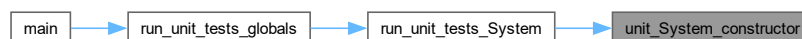
```
void unit_System_constructor (
    void )
```

Testa o construtor parametrizado da classe [System](#).

Verifica se, ao criar sistemas com valores iniciais distintos, o valor armazenado corresponde ao informado na construção. Here is the call graph for this function:



Here is the caller graph for this function:

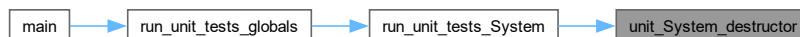


8.33.2.3 unit_System_destructor()

```
void unit_System_destructor (  
    void )
```

Testa o destrutor da classe [System](#).

Verifica se o objeto é destruído sem erros ou vazamentos de memória. Here is the caller graph for this function:



8.33.2.4 unit_System_getValue()

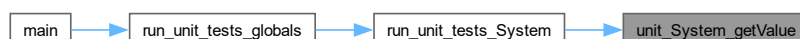
```
void unit_System_getValue (  
    void )
```

Testa o método `getValue()` da classe [System](#).

Verifica se o valor retornado corresponde ao valor definido na construção do objeto. Here is the call graph for this function:



Here is the caller graph for this function:



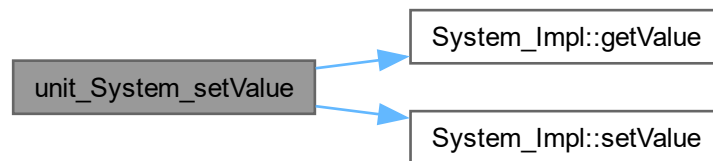
8.33.2.5 unit_System_setValue()

```
void unit_System_setValue (  
    void )
```

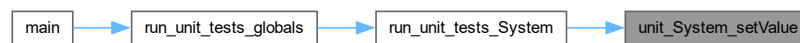
Testa o método `setValue()` da classe [System](#).

Verifica se o valor do sistema é atualizado corretamente após a chamada de `setValue()`, sendo refletido

por uma chamada subsequente a `getValue()`. Here is the call graph for this function:



Here is the caller graph for this function:



8.34 unit_system.h

[Go to the documentation of this file.](#)

```

00001 #ifndef UNIT_SYSTEM
00002 #define UNIT_SYSTEM
00003
00012
00013 #include "../include/system_impl.h"
00014
00021 void unit_System_constructor(void);
00022
00028 void unit_System_destructor(void);
00029
00036 void unit_System_getValue(void);
00037
00045 void unit_System_setValue(void);
00046
00053 void run_unit_tests_System(void);
00054
00055 #endif
  
```

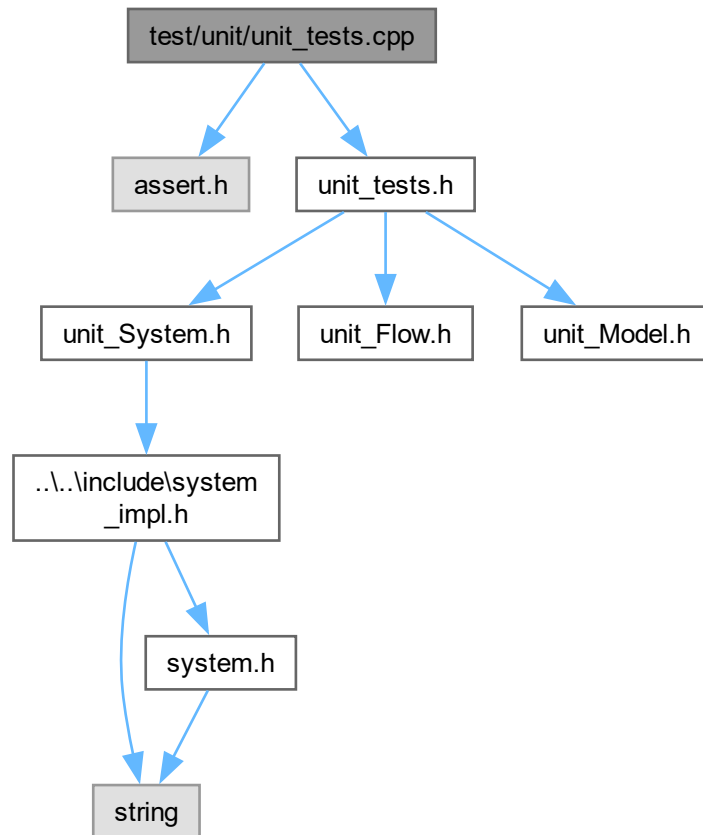
8.35 test/unit/unit_tests.cpp File Reference

Implementação da função orquestradora dos testes unitários globais.

```

#include <assert.h>
#include "unit_tests.h"
  
```

Include dependency graph for unit_tests.cpp:



Functions

- void [run_unit_tests_globals](#) (void)
Executa todos os testes unitários do projeto em sequência.

8.35.1 Detailed Description

Implementação da função orquestradora dos testes unitários globais.

Contém a implementação de [run_unit_tests_globals\(\)](#), que centraliza a execução de todos os testes unitários do projeto, chamando as suítes de teste de cada classe individualmente.

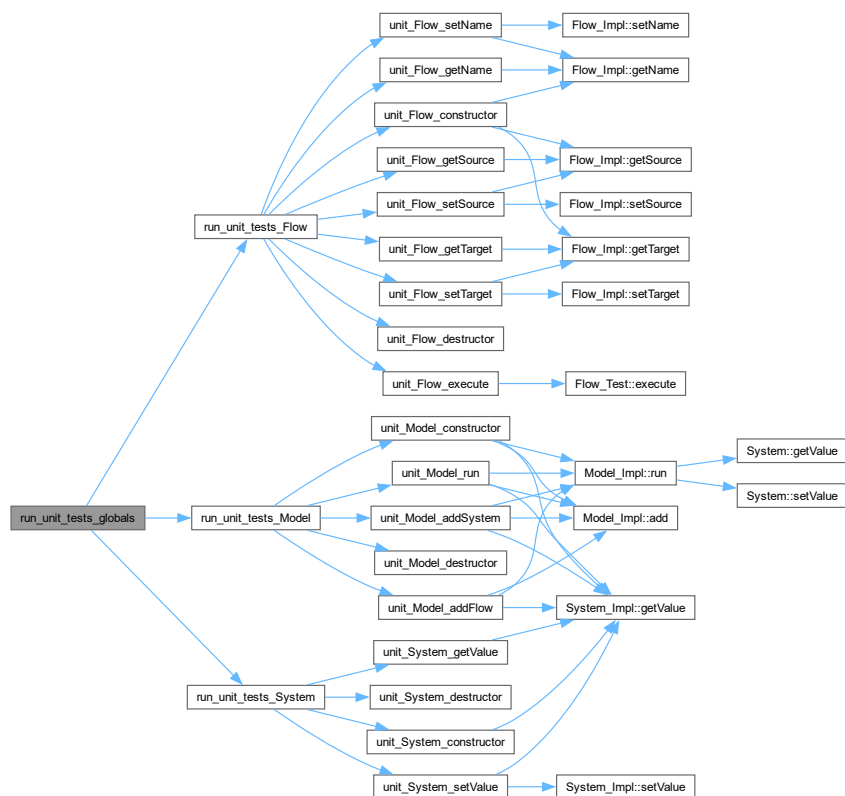
8.35.2 Function Documentation

8.35.2.1 run_unit_tests_globals()

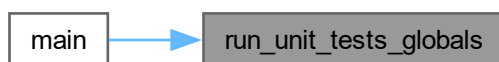
```
void run_unit_tests_globals (
    void )
```

Executa todos os testes unitários do projeto em sequência.

Chama [run_unit_tests_System\(\)](#), [run_unit_tests_Flow\(\)](#) e [run_unit_tests_Model\(\)](#), cobrindo os testes unitários de todas as classes do modelo de simulação. Here is the call graph for this function:



Here is the caller graph for this function:

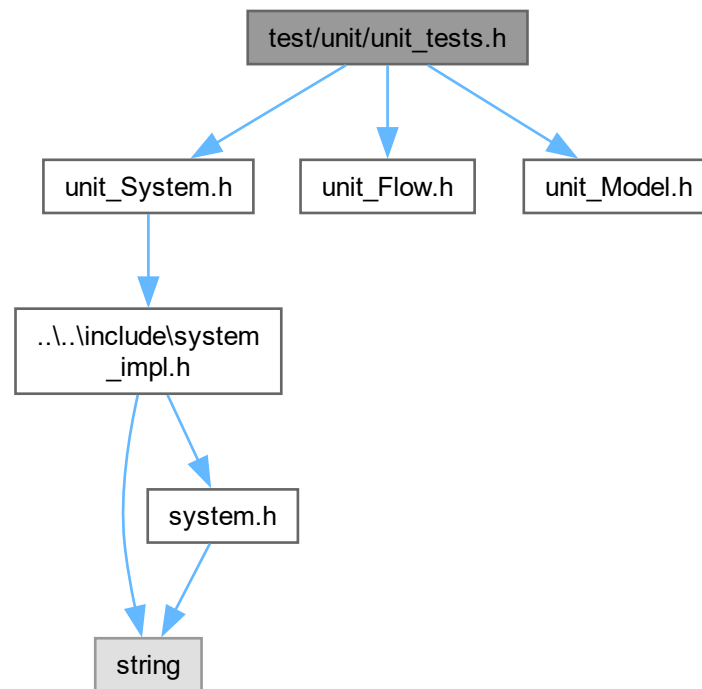


8.36 test/unit/unit_tests.h File Reference

Declaração da função de entrada dos testes unitários globais.

```
#include "unit_System.h"
#include "unit_Flow.h"
#include "unit_Model.h"
```

Include dependency graph for unit_tests.h:



Functions

- void [run_unit_tests_globals](#) (void)
Executa todos os testes unitários do projeto em sequência.

8.36.1 Detailed Description

Declaração da função de entrada dos testes unitários globais.

Este arquivo agrega os cabeçalhos de todos os módulos de teste unitário ([System](#), [Flow](#) e [Model](#)) e declara a função orquestradora que os executa em sequência.

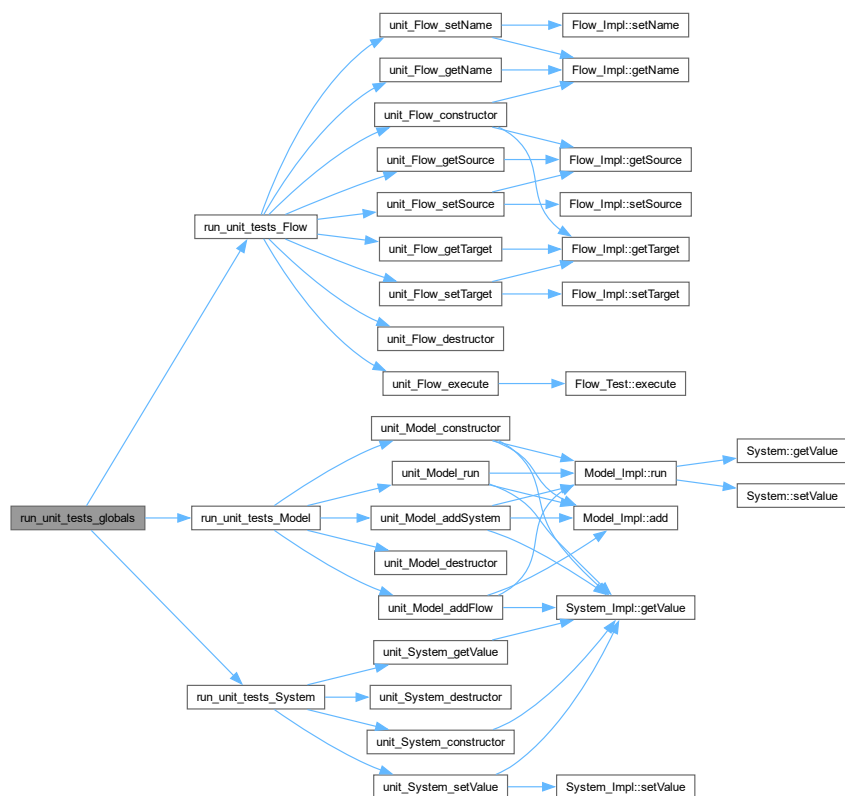
8.36.2 Function Documentation

8.36.2.1 [run_unit_tests_globals](#)()

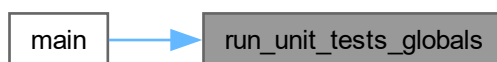
```
void run_unit_tests_globals (
    void )
```

Executa todos os testes unitários do projeto em sequência.

Chama [run_unit_tests_System\(\)](#), [run_unit_tests_Flow\(\)](#) e [run_unit_tests_Model\(\)](#), cobrindo os testes unitários de todas as classes do modelo de simulação. Here is the call graph for this function:



Here is the caller graph for this function:



8.37 unit_tests.h

[Go to the documentation of this file.](#)

```

00001 #ifndef UNIT_TESTS
00002 #define UNIT_TESTS
00003
00012
00013 #include "unit_System.h"
00014 #include "unit_Flow.h"
00015 #include "unit_Model.h"
00016
00024 void run_unit_tests_globals(void);
00025
00026 #endif
  
```

Index

- ~Flow
 - Flow, [16](#)
- ~FlowComplexo
 - FlowComplexo, [35](#)
- ~FlowExponencial
 - FlowExponencial, [41](#)
- ~FlowLogistico
 - FlowLogistico, [47](#)
- ~Flow_Impl
 - Flow_Impl, [23](#)
- ~Model
 - Model, [54](#)
- ~Model_Impl
 - Model_Impl, [58](#)
- ~System
 - System, [64](#)
- ~System_Impl
 - System_Impl, [70](#)
- add
 - Model, [54](#)
 - Model_Impl, [59](#)
- complexFuncionalTest
 - functional_test.cpp, [95](#)
 - functional_test.h, [80](#)
- execute
 - Flow, [17](#)
 - Flow_Impl, [23](#)
 - Flow_Test, [30](#)
 - FlowComplexo, [35](#)
 - FlowExponencial, [41](#)
 - FlowLogistico, [47](#)
 - FlowTest, [52](#)
- exponentialFuncionalTest
 - functional_test.cpp, [96](#)
 - functional_test.h, [81](#)
- floatingPointComparison
 - functional_test.cpp, [97](#)
- Flow, [15](#)
 - ~Flow, [16](#)
 - execute, [17](#)
 - getName, [17](#)
 - getSource, [17](#)
 - getTarget, [17](#)
 - setName, [17](#)
 - setSource, [17](#)
 - setTarget, [18](#)
- Flow_Impl, [18](#)
 - ~Flow_Impl, [23](#)
 - execute, [23](#)
 - Flow_Impl, [21](#), [22](#)
 - getName, [23](#)
 - getSource, [23](#)
 - getTarget, [24](#)
 - name, [26](#)
 - operator=, [24](#)
 - setName, [25](#)
 - setSource, [25](#)
 - setTarget, [26](#)
 - source, [26](#)
 - target, [26](#)
- Flow_Test, [26](#)
 - execute, [30](#)
 - Flow_Test, [29](#), [30](#)
- FlowComplexo, [31](#)
 - ~FlowComplexo, [35](#)
 - execute, [35](#)
 - FlowComplexo, [34](#)
 - operator=, [35](#)
- FlowExponencial, [36](#)
 - ~FlowExponencial, [41](#)
 - execute, [41](#)
 - FlowExponencial, [40](#)
 - operator=, [41](#)
- FlowLogistico, [42](#)
 - ~FlowLogistico, [47](#)
 - execute, [47](#)
 - FlowLogistico, [46](#)
 - operator=, [47](#)
- flows
 - Model_Impl, [62](#)
- FlowTest, [48](#)
 - execute, [52](#)
 - FlowTest, [51](#), [52](#)
- functional_test.cpp
 - complexFuncionalTest, [95](#)
 - exponentialFuncionalTest, [96](#)
 - floatingPointComparison, [97](#)
 - logisticalFuncionalTest, [97](#)
- functional_test.h
 - complexFuncionalTest, [80](#)
 - exponentialFuncionalTest, [81](#)
 - logisticalFuncionalTest, [81](#)
- getName
 - Flow, [17](#)
 - Flow_Impl, [23](#)

- System, [64](#)
 - System_Impl, [70](#)
- getSource
 - Flow, [17](#)
 - Flow_Impl, [23](#)
- getTarget
 - Flow, [17](#)
 - Flow_Impl, [24](#)
- getValue
 - System, [64](#)
 - System_Impl, [70](#)
- include Directory Reference, [11](#)
- include/flow.h, [75](#)
- include/flow_impl.h, [76](#)
- include/flows.h, [77](#), [78](#)
- include/functional_test.h, [79](#), [82](#)
- include/model.h, [82](#), [83](#)
- include/model_impl.h, [83](#), [84](#)
- include/system.h, [85](#)
- include/system_impl.h, [85](#), [86](#)
- logisticalFuncionalTest
 - functional_test.cpp, [97](#)
 - functional_test.h, [81](#)
- MAIN
 - main.cpp, [88](#)
- main
 - main.cpp, [88](#), [89](#), [91](#)
- main.cpp
 - MAIN, [88](#)
 - main, [88](#), [89](#), [91](#)
 - MAIN_FUNCIONAL_TESTS, [89](#)
- MAIN_FUNCIONAL_TESTS
 - main.cpp, [89](#)
- Model, [52](#)
 - ~Model, [54](#)
 - add, [54](#)
 - run, [54](#)
 - showModel, [55](#)
- Model_Impl, [55](#)
 - ~Model_Impl, [58](#)
 - add, [59](#)
 - flows, [62](#)
 - Model_Impl, [58](#)
 - operator=, [60](#)
 - run, [60](#)
 - showModel, [62](#)
 - systems, [62](#)
- MyVensim, [1](#)
- name
 - Flow_Impl, [26](#)
 - System_Impl, [72](#)
- operator=
 - Flow_Impl, [24](#)
 - FlowComplexo, [35](#)
 - FlowExponencial, [41](#)
 - FlowLogistico, [47](#)
 - Model_Impl, [60](#)
 - System_Impl, [71](#)
- README.md, [87](#)
- run
 - Model, [54](#)
 - Model_Impl, [60](#)
- run_unit_tests_Flow
 - unit_flow.cpp, [100](#)
 - unit_flow.h, [106](#)
- run_unit_tests_globals
 - unit_tests.cpp, [129](#)
 - unit_tests.h, [131](#)
- run_unit_tests_Model
 - unit_model.cpp, [113](#)
 - unit_model.h, [117](#)
- run_unit_tests_System
 - unit_system.cpp, [122](#)
 - unit_system.h, [125](#)
- setName
 - Flow, [17](#)
 - Flow_Impl, [25](#)
 - System, [65](#)
 - System_Impl, [72](#)
- setSource
 - Flow, [17](#)
 - Flow_Impl, [25](#)
- setTarget
 - Flow, [18](#)
 - Flow_Impl, [26](#)
- setValue
 - System, [65](#)
 - System_Impl, [72](#)
- showModel
 - Model, [55](#)
 - Model_Impl, [62](#)
- source
 - Flow_Impl, [26](#)
- src Directory Reference, [12](#)
- src/flow_impl.cpp, [87](#)
- src/main.cpp, [87](#)
- src/model_impl.cpp, [92](#)
- src/system_impl.cpp, [93](#)
- System, [63](#)
 - ~System, [64](#)
 - getName, [64](#)
 - getValue, [64](#)
 - setName, [65](#)
 - setValue, [65](#)
- System_Impl, [66](#)
 - ~System_Impl, [70](#)
 - getName, [70](#)
 - getValue, [70](#)
 - name, [72](#)
 - operator=, [71](#)
 - setName, [72](#)

- setValue, [72](#)
 - System_Impl, [69](#)
 - value, [72](#)
- systems
 - Model_Impl, [62](#)
- target
 - Flow_Impl, [26](#)
- test Directory Reference, [12](#)
- test/functional Directory Reference, [11](#)
- test/functional/flows.cpp, [93](#)
- test/functional/functional_test.cpp, [94](#)
- test/functional/main.cpp, [88](#)
- test/unit Directory Reference, [13](#)
- test/unit/main.cpp, [90](#)
- test/unit/unit_flow.cpp, [98](#)
- test/unit/unit_flow.h, [105](#), [111](#)
- test/unit/unit_model.cpp, [111](#)
- test/unit/unit_model.h, [116](#), [120](#)
- test/unit/unit_system.cpp, [121](#)
- test/unit/unit_system.h, [124](#), [128](#)
- test/unit/unit_tests.cpp, [128](#)
- test/unit/unit_tests.h, [130](#), [132](#)
- unit_flow.cpp
 - run_unit_tests_Flow, [100](#)
 - unit_Flow_constructor, [100](#)
 - unit_Flow_destructor, [101](#)
 - unit_Flow_execute, [101](#)
 - unit_Flow_getName, [102](#)
 - unit_Flow_getSource, [102](#)
 - unit_Flow_getTarget, [103](#)
 - unit_Flow_setName, [103](#)
 - unit_Flow_setSource, [104](#)
 - unit_Flow_setTarget, [104](#)
- unit_flow.h
 - run_unit_tests_Flow, [106](#)
 - unit_Flow_constructor, [106](#)
 - unit_Flow_destructor, [107](#)
 - unit_Flow_execute, [107](#)
 - unit_Flow_getName, [108](#)
 - unit_Flow_getSource, [108](#)
 - unit_Flow_getTarget, [109](#)
 - unit_Flow_setName, [109](#)
 - unit_Flow_setSource, [110](#)
 - unit_Flow_setTarget, [110](#)
- unit_Flow_constructor
 - unit_flow.cpp, [100](#)
 - unit_flow.h, [106](#)
- unit_Flow_destructor
 - unit_flow.cpp, [101](#)
 - unit_flow.h, [107](#)
- unit_Flow_execute
 - unit_flow.cpp, [101](#)
 - unit_flow.h, [107](#)
- unit_Flow_getName
 - unit_flow.cpp, [102](#)
 - unit_flow.h, [108](#)
- unit_Flow_getSource
 - unit_flow.cpp, [102](#)
 - unit_flow.h, [108](#)
- unit_flow.cpp, [102](#)
- unit_flow.h, [108](#)
- unit_Flow_getTarget
 - unit_flow.cpp, [103](#)
 - unit_flow.h, [109](#)
- unit_Flow_setName
 - unit_flow.cpp, [103](#)
 - unit_flow.h, [109](#)
- unit_Flow_setSource
 - unit_flow.cpp, [104](#)
 - unit_flow.h, [110](#)
- unit_Flow_setTarget
 - unit_flow.cpp, [104](#)
 - unit_flow.h, [110](#)
- unit_model.cpp
 - run_unit_tests_Model, [113](#)
 - unit_Model_addFlow, [113](#)
 - unit_Model_addSystem, [114](#)
 - unit_Model_constructor, [114](#)
 - unit_Model_destructor, [115](#)
 - unit_Model_run, [115](#)
 - unit_Model_showModel, [116](#)
- unit_model.h
 - run_unit_tests_Model, [117](#)
 - unit_Model_addFlow, [117](#)
 - unit_Model_addSystem, [118](#)
 - unit_Model_constructor, [119](#)
 - unit_Model_destructor, [119](#)
 - unit_Model_run, [119](#)
 - unit_Model_showModel, [120](#)
- unit_Model_addFlow
 - unit_model.cpp, [113](#)
 - unit_model.h, [117](#)
- unit_Model_addSystem
 - unit_model.cpp, [114](#)
 - unit_model.h, [118](#)
- unit_Model_constructor
 - unit_model.cpp, [114](#)
 - unit_model.h, [119](#)
- unit_Model_destructor
 - unit_model.cpp, [115](#)
 - unit_model.h, [119](#)
- unit_Model_run
 - unit_model.cpp, [115](#)
 - unit_model.h, [119](#)
- unit_Model_showModel
 - unit_model.cpp, [116](#)
 - unit_model.h, [120](#)
- unit_system.cpp
 - run_unit_tests_System, [122](#)
 - unit_System_constructor, [122](#)
 - unit_System_destructor, [123](#)
 - unit_System_getValue, [123](#)
 - unit_System_setValue, [124](#)
- unit_system.h
 - run_unit_tests_System, [125](#)
 - unit_System_constructor, [126](#)
 - unit_System_destructor, [126](#)

- [unit_System_getValue](#), [127](#)
 - [unit_System_setValue](#), [127](#)
- [unit_System_constructor](#)
 - [unit_system.cpp](#), [122](#)
 - [unit_system.h](#), [126](#)
- [unit_System_destructor](#)
 - [unit_system.cpp](#), [123](#)
 - [unit_system.h](#), [126](#)
- [unit_System_getValue](#)
 - [unit_system.cpp](#), [123](#)
 - [unit_system.h](#), [127](#)
- [unit_System_setValue](#)
 - [unit_system.cpp](#), [124](#)
 - [unit_system.h](#), [127](#)
- [unit_tests.cpp](#)
 - [run_unit_tests_globals](#), [129](#)
- [unit_tests.h](#)
 - [run_unit_tests_globals](#), [131](#)
- [value](#)
 - [System_Impl](#), [72](#)